Model Context Protocol — The Why

MCP Trilogy — Part 1 · CampusX

https://youtu.be/Zmy439spZB4

Duration: 52:00 · Transcript: Whisper small (862 segments) · Frames: 50 scene-change extracts

This explainer was generated automatically from the CampusX lecture *Model Context Protocol — The Why* by extracting scene-change frames from the video and aligning them with a Whisper-transcribed audio track. Each section below pairs a frame from the lecture with the speech delivered around that point in time. The frames were chosen by ffmpeg scene-detection at a low (0.10) threshold so slide transitions, code views, and demos are all captured.

Section 1 — 00:00 – 02:17

Figure 1. Frame at 00:00.

Hi guys, my name is Nitish and you're welcome to my YouTube channel. So if you've watched my last video, you'll remember that I've started a new playlist on this channel. And the name of this playlist is Model Context Protocol, MCP. And the motivation behind starting this playlist is that this particular term, MCP, has become very popular in the last 1 year. And it seems like in the next 3 to 5 years MCP will become an industry standard. Which basically means that in the industry, everyone will have to integrate MCP in their software. And that is why I received a lot of messages and responses from you guys that please cover MCP. So that is why in the last 30 to 40 days, I was studying MCP. I was trying to implement it on my laptop. And eventually I planned a curriculum. Now, according to that curriculum, I have decided that I will put 3 videos in this playlist. The first video will be The Why, the second will be The What, the third will be The How. And my goal is to teach you MCP in this way through these 3 videos. Now, if we talk about today's video, then today's video is the first video of this playlist. And in this, we will cover the Why aspect of MCP. So basically, my goal is in this video that I want to teach you in this way that why MCP needs to be studied. What is the problem that MCP solves? And to explain the whole thing, I am going to take a very storytelling approach. I will start from the beginning when 4GPT came into the picture. And from there to today's date, whatever has happened, I will tell you all those things in a story-like fashion. And this will benefit that I will be able to develop an intuition deeply in your mind that why MCP needs to be as a technology. Okay, so I really hope I could explain to you in the first minute what is our goal in this video. And now if you think everything is right, everything is sounding right, then now let's start the video. So guys, our story starts on November 30, 2022.

Section 2 — 02:17 – 09:36

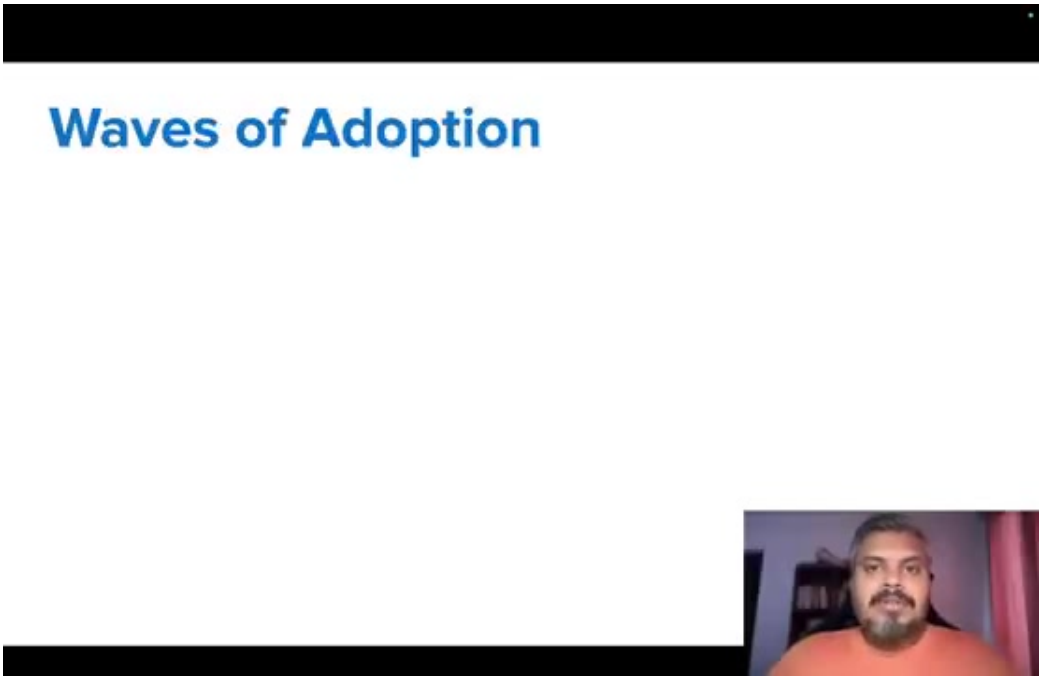

Figure 2. Frame at 04:35.

So guys, our story starts on November 30, 2022. The day 4GPT was released. And within five days, 4GPT crossed the mark of 1 million users. And in the next two months, it crossed the mark of 100 million users. And these are not small numbers. In fact, before 4GPT, no software had achieved such crazy numbers. Whether you talk about Google or Facebook or Twitter. In fact, 4GPT is a completely different class of software. Such software that never came before. And 4GPT gives you such a capability that no software would have given you before. And that capability is that you can talk to machines just like you talk to a human being. In your natural language. Think about it. The relationship with our machines is probably 500-600 years old. At first, mechanical machines used to happen. Then electrical machines came. And then in the last 50 years, we have been working a lot with computers. But in these 500-600 years, our relationship is mostly transactional. Which basically means that if I want anything from my machine, then I just perform an action. And I get the results of it. I feel hot. I want to switch on the fan. I want to do some calculations. I want to press the buttons on the calculator. I want to fill a form. I will simply type things on the keyboard and press a button. And the machine will do our job. This is a transactional relationship. Where we talk very minimally and get our work done. But after 4GPT, you can talk to computers just like you talk to a human being. You can express yourself if you want. You can express yourself in front of the machine. You can engage in a thoughtful discussion with your computer. In fact, you can make it your work partner. After 4GPT, all this happened. That is why I am saying that 4GPT is a completely different class of software. Now, like any other software, when 4GPT was launched, its adoption gradually happened. In fact, if you ask me, then the adoption of 4GPT happened in 3 stages. So, the first stage, I am calling it the stage of pure wonder. I still remember, around December 1st week, a student messaged me on WhatsApp. And he said, Sir, try out this particular chatbot. It's crazy good. And I still remember, the next day, I was talking to 4GPT. And I was shocked how intelligent this chatbot is. And I am pretty sure you all must have experienced the same thing when you first talked to 4GPT. So, in this whole stage, I think people did only one thing. And that was to calm down their curiosity. So basically, people asked 4GPT about Utpatang. For example, I am pretty sure someone must have asked to explain quantum physics to a cat perspective. Or someone else must have asked, if gravity turns upside down, what will

happen? Or someone must have asked, write a song on a pizza in Shakespeare's style. And basically, we were asking these crazy questions and 4GPT was responding intelligently. And we took a screenshot of it and posted it on LinkedIn, Instagram, WhatsApp. So in short, in this first wave of adoption, what happened was that the social media exploded. And we were constantly getting screenshots of what people asked and what 4GPT replied. So, there was no meaningful work here. But our curiosity calmed down a bit. Then comes the second wave of adoption. I am calling it a professional adoption because once the initial period was over, our curiosity calmed down a bit and we understood 4GPT. So, we got a question automatically that what we are doing with 4GPT is fine. But is this chatbot so intelligent that we could help in our professional work? So, this was the first time when a lawyer put a 50-page contract in 4GPT and asked us to summarize it. And 4GPT did it. Maybe this was the first time when a coder pasted his code and asked if there is an error here. Can you debug this? And 4GPT did it. Or a teacher like me said that I have to plan a curriculum and I don't want to study everything. Can you do this for me? And 4GPT did it again. So, this was the first time when we collectively realized that this is not just a tool to joke. In fact, this chatbot has got serious potential and it can become our work partners. I mean, we can double our productivity with the help of this software. And this happened where you used to take 6 hours to do some work. With the help of 4GPT, you could do that work in 3 hours. So, on the whole, after this wave of adoption, in the whole world, people saw a productivity boom. Collectively, everyone's productivity increased. Everyone's way of working became easier. And here, 4GPT showed its true power. And then comes the third wave of adoption, which is the API revolution. At this particular stage, OpenAI did not just release 4GPT, they also released APIs for the general public. They wanted to say that if you want, you can integrate chatting capabilities like 4GPT in your existing software. And this happened too. Many companies realized that although our software is very good, it would be great if we could somehow get intelligent features like 4GPT in our software. And this was the first time when Microsoft advanced and added a co-pilot feature in Word, Excel, PowerPoint, which was basically a way to communicate with their software in a natural way. Then Google came and integrated AI in Gmail, Doc, Sheets and Drive. And this was the first time when new age tools started coming, like cursor or perplexity. And what happened was that AI was not restricted to just 4GPT. The software that we were already using, also had AI capabilities. So in short, after this wave of adoption, AI became a little more accessible. If I need AI, then it is not only available in 4GPT,

Section 3 — 09:36 – 15:49

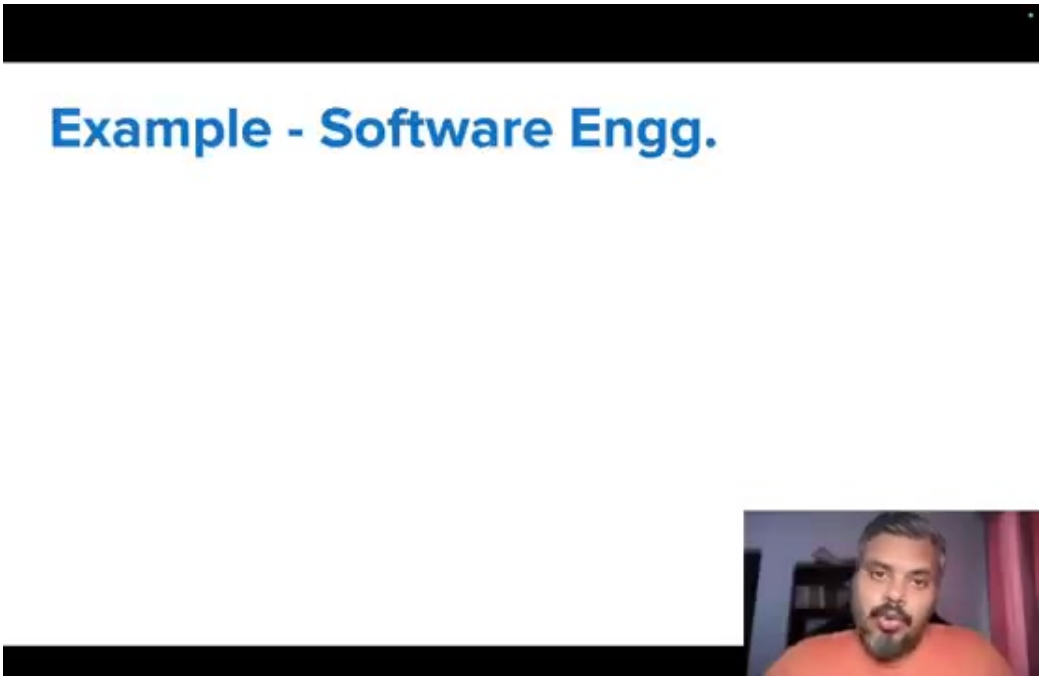

Figure 3. Frame at 14:36.

If I need AI, then it is not only available in 4GPT, it is also available in different software around me. So in these three waves, you can say that LLMs came into our world completely. So after coming to 4GPT and its APIs, all the software around us got AI enabled. And along with that, it also made sure that AI became very accessible. But with this accessibility, a new problem emerged. And I call this problem the problem of fragmentation. So I mean to say that we have a lot of software that we use on a daily basis. And after coming to 4GPT and its APIs, all those software got AI enabled. For example, we use Notion, we also use Slack, and these are both AI enabled. But the problem is that the AI of Notion has no idea what is going on in Slack's AI. Or if we use VS Code, and we have installed a coding assistant there, then this coding assistant has no idea what is going on in Microsoft Teams. So basically, all of a sudden, we realized that we are living in multiple AI worlds. One AI world of Notion, one AI world of Slack, one AI world of our VS Code coding assistant, and so on. And if we have to do a small task, then we have to juggle between these multiple AI worlds. Some information is read at one place, some information is read at another place. And our job is to go to all those places and merge that information to get our job done. Now think about it. We never wanted this problem to be faced. Our vision when we first saw 4GPT and used it, our vision was that we wish we get a unified AI agent who understands our entire work end-to-end. And not only understand, but when I get stuck in some problem, then that unified agent can solve any problem of mine. This was our vision. But instead, what did we get? We got 5 different AI tools. Some work with this AI tool, some work with other AI tools, and so on. Now, the unified agent that we had to make, this work of making is actually not easy. And the biggest problem on the way to make this unified agent, is the problem of context. Which we will understand next. Now let's talk about context. Context is a very fundamental concept in MCP. In fact, it is so fundamental that context comes in the name of MCP. Model-Context Protocol. So let's take some time to understand this concept in detail. First of all, let's understand in simple words what context is. So, in simplest words, context is everything and AI can see, when it generates a response. When AI generates a response, the time generated by generating that response, the things that are visible to it, we call it context. A little formal definition would be, that context refers to the information that the LLM uses to generate a response. Now that information, conversation, history can also be,

external documents can also be. Let's understand through an example. Suppose you are talking to a chat GPT about quantum physics. You asked what quantum physics is, where to learn quantum physics, what are the good books. Now suddenly you asked how difficult it is to learn a particular topic. Now how do you know about the chat GPT? The answer is conversation history. The chat GPT will go suddenly and see the whole conversation history, and he will realize that the user is talking about quantum physics. So he will understand that he is asking how difficult it is to learn quantum physics, and he will answer you instantly. So in this particular scenario, your conversation history is the context of your AI model. Okay? While typing the next response, your AI can see the whole conversation history, and it can print its response with the help of it. Okay? So I hope you understood the context roughly. Now in this particular example, context is shown in a very simple way. Basically in the form of conversation history. But context is not necessary to be so simple. Especially if you talk about professional use cases. So now I will take you to another example and explain how difficult the context can be to understand. So let's talk about a software engineer. And what is going on in his life in particular? Let's talk about that. And let's understand the context there. Okay? So I am a software developer, and I work in a startup. We have a product. Suppose there is an edtech product where people purchase courses, just like you Demi. Now suddenly, I was given a requirement, and I was told that I have to add a two-factor authentication feature in our website so that our website becomes a little more secure. So I will tell you how this whole thing works. So what will happen first? A ticket will be raised on a software like Gira. And that ticket will become easy for me. That I have to develop this feature. Now, what will I do to develop this feature? First, I will go to GitHub. And from there, I will pull the most updated code base on my machine. Right? And then, obviously two-factor authentication is required. So for that, I will have to go to the database and understand how our existing schema looks like.

Section 4 — 15:49 – 17:42

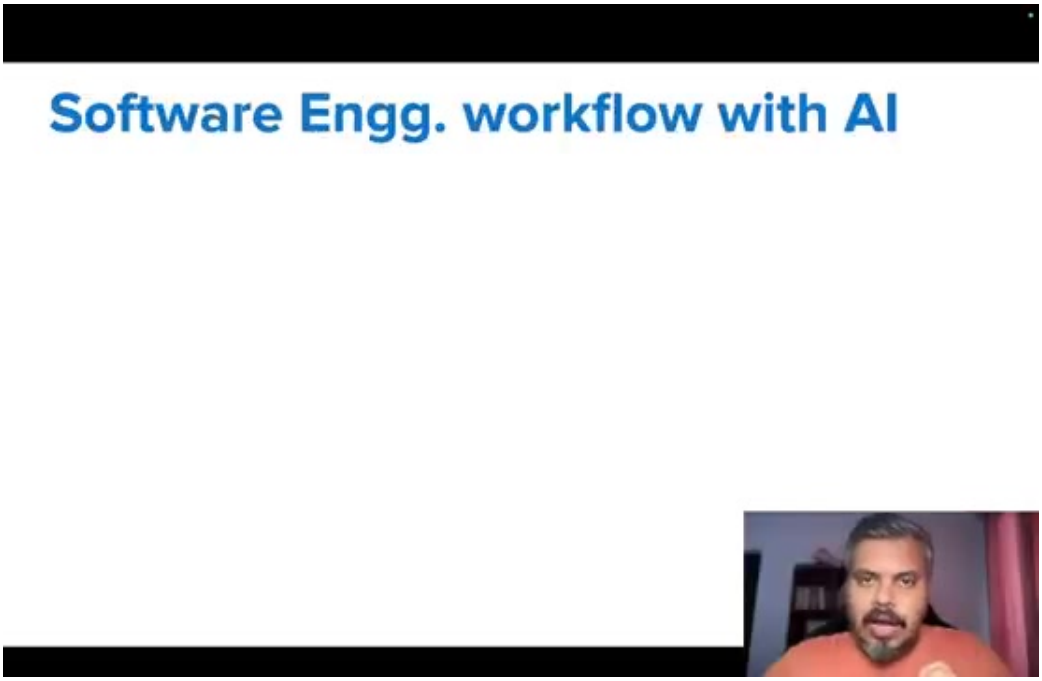

Figure 4. Frame at 17:03.

So for that, I will have to go to the database and understand how our existing schema looks like. So I will use a software like MySQL which our company is using. And by entering that software, I will study the schema of the database. Suddenly, I will remember that I have to follow certain security guidelines in order to implement the two-factor authentication. So what will I do? I will go to Google Drive. And from there, I will pick up a security document with which I will implement this whole two-factor authentication. Finally, if I go somewhere first, then I will go and talk to my teammate or help with the help of Slack like a software. So in this particular case, if you can see the two-factor authentication system that I have to develop is my task. But the context related to this task is existing in many places. Now, it is not so simple as in our last example, our whole context was existing in just one conversation history. In this scenario, our context is existing in multiple places. Now think about it yourself. If I have to develop this two-factor authentication with the help of AI, let's say 4GPT, then how will I approach it? First of all, I will go to Jira. From there, whatever is written in the ticket, how to implement this whole solution, I will copy it and paste it on 4GPT. After that, I will go to the code base. I will copy the code of 10-12 files, which is our existing authentication system. I will paste the code in 4GPT. After that, I will go to MySQL. I will copy the schema and paste it on 4GPT.

Section 5 — 17:42 – 20:50

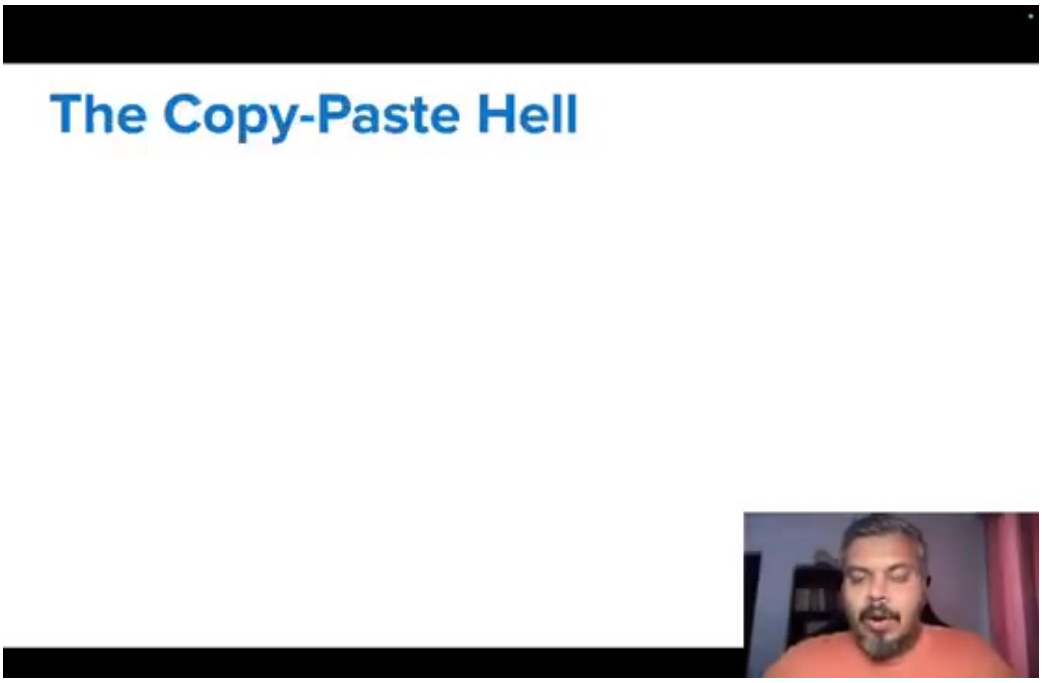

Figure 5. Frame at 18:21.

I will copy the schema and paste it on 4GPT. After that, I will send my security document, the specifications written there, I will paste it on 4GPT. After that, if there are any discussions in Slack, I will copy it and paste it on 4GPT. And finally, after pasting so much, after 20 minutes, I will ask my first question. How to implement two-factor authentication in such a system? Now I guess you can see that the context is scattered. And making the context is so difficult and time-taking. And this is the biggest problem. Our problem is that before asking a simple question, we have to copy thousands of lines of code and paste it on 4GPT. Meaning, if you say in a way, then we developers have become human APIs. And what is our job? Our job is to assemble the context for software like 4GPT. So we are basically human APIs.

And in a way, it is not wrong to say that the context assembly we are doing, we are spending more time in this rather than actually developing the product. And apart from this, you have to take care of the context provided to AI. What is left, what is forgotten, what is summarized. This whole thing is not scalable at all. Think about it, if you have a codebase of 50,000 lines, then can you summarize it for your AI? Or can you copy and paste the entire context in 4GPT? So it's not at all possible that you assemble the context from different places and give your AI. And that is why I am saying that it is very difficult to make a unified AI agent. Because the biggest challenge in front of us is the context assembly. In a real professional scenario, your context exists across systems. And to be able to paste all those things on a copy and paste them in one place and then be able to talk to 4GPT or any AI software is a very very difficult thing to do. And at scale, it's not at all possible. So if I summarize this context problem, the main problem is that if we want our AI chatbot to be able to understand all the things related to our work and be able to help us solve problems, then it is important that it be able to see all our work. Basically, all our work should be a part of its context. But the problem is that our context, our work is scattered. So then showing AI all that work has become a very hard work because we are manually copying and pasting things. And as your project grows, the more tools you work with,

Section 6 — 20:50 – 23:45

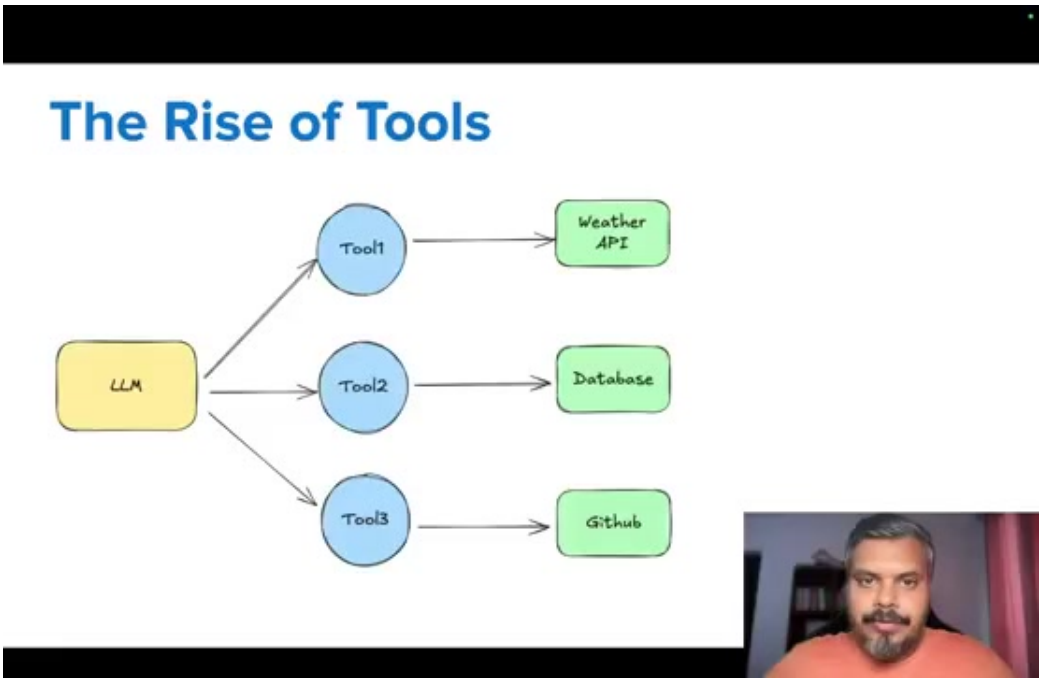

Figure 6. Frame at 23:19.

And as your project grows, the more tools you work with, the things you copy and paste will go on failing. Ideally, it would have been good if somehow a software like ChatGPT could go to all the places and fetch the necessary context. So then we don't have to do the manual copy and paste work. And guess what? This problem was solved eventually. So what happened? OpenAI released a new concept mid of 2023, whose name was Function Calling. Now this was a very simple but very powerful concept. With the help of Function Calling, you can call an external function from your LLM. Which basically means that your LLM will not only be useful for chatting, but if needed, it can also complete a task. So in Function Calling, you provide some set of functions to your LLM. And you give a description about every function to LLM that

this particular function does this. Now guess what? We gave our LLM a function called LoadFile. And we gave this description that if in the future, the user wants to load a file, then you can use this function. So what will happen now? When the user gives such a prompt, read the content of the file abc.txt, then instead of normal chatting, the LLM will understand that the user is telling him to do something. So what will he do from the start? He will scan the list of functions. And whatever function's description matches with the given task, automatically what LLM will do? He will call that function. With the right set of arguments, like here he will say to call LoadFile with this argument abc.txt. And then what will we do? We will call this LoadFile function with this input. And our task will be executed. Now this is a very small concept. But this was path breaking. Because for the first time, now you can't just chat with LLM, but at the same time you can execute a task as well. So then some such architecture came into the picture that there is one LLM with which we have connected a lot of functions, connected a lot of tools. There is one tool to fetch data from the weather API, there is one tool to run queries in the database, there is another tool to fetch information from GitHub, there is another tool to search the web.

Section 7 — 23:45 – 25:20

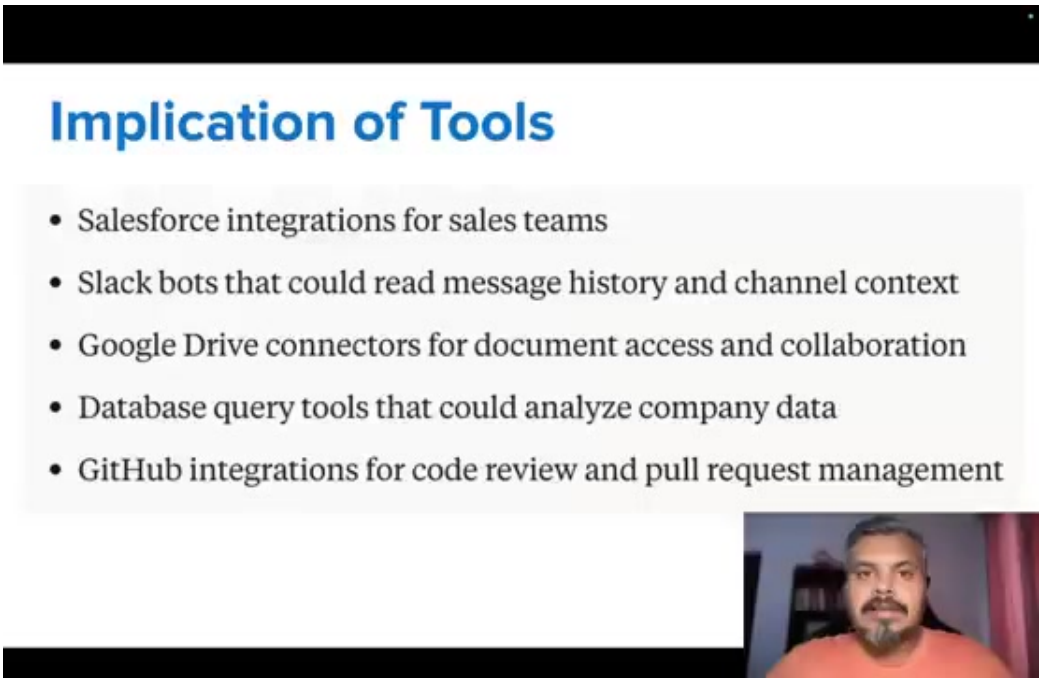

Figure 7. Frame at 24:12.

there is another tool to search the web. So basically you can create different functions for your different tasks. And every function internally will tell you how to execute the given task. And your LLM has only one thing, that the user's prompt will read it and understand which tool to invoke. And this was a very revolutionary idea. After it came, tools became very good. Any company that was already working with LLMs, they realized the power of the tools, and they started making a lot of tools to make their workflow seamless, to make the context assembly seamless. For example, they first made tools for enterprise software. For example, every company uses your Salesforce, you use Slack, you use Google Drive, you use databases. So first of all, the company told its developers that you do one thing, you make a tool for our AI chatbot for Salesforce integration. Similarly, you make a Slackbot tool for Slack. You make connectors of Google Drive, you make tools to run queries on the database, or you make a tool for

integration of GitHub, so that all these places where our context exists, we can connect them and fetch the context at any time. Then companies also made internal tools. For example, HR departments made different functions, tools to access the employees' data,

Section 8 — 25:20 – 27:57

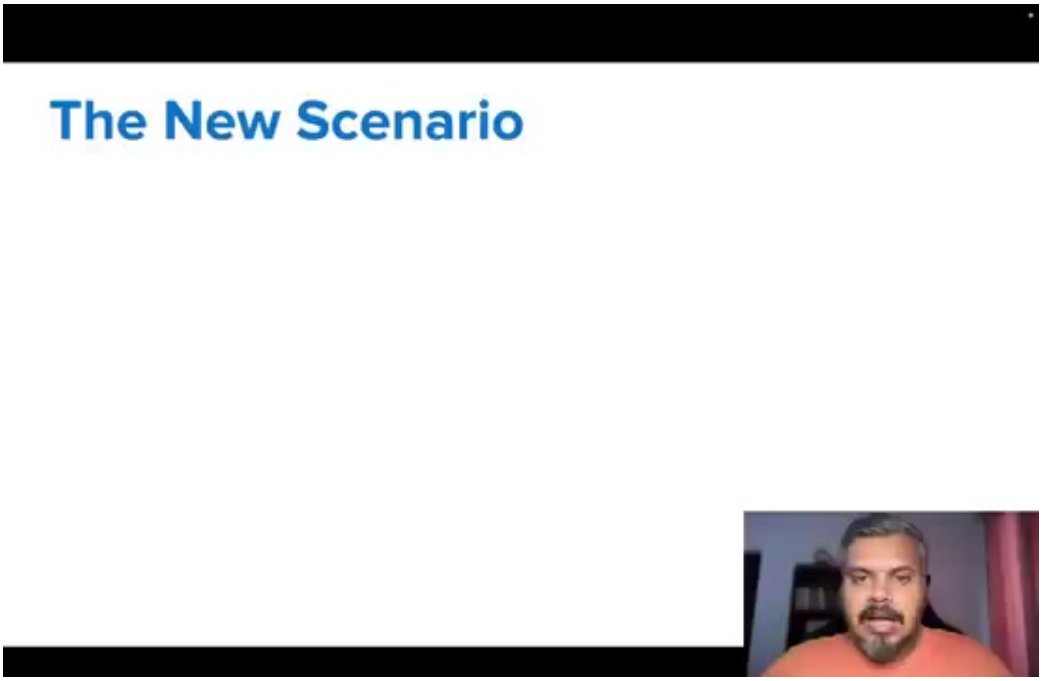

Figure 8. Frame at 26:28.

For example, HR departments made different functions, tools to access the employees' data, the finance team made tools for accounting systems, the marketing team made tools for campaign management, the IT department made tools for infrastructure management. And at this point, the new AI first software, they also started making a lot of tools. For example, Cursor, our AI powered code editor, he made a tool for file system access, so that you can access any file on your local file system, and you can search it intelligently. Similarly, Perplexity, he made a tool for web browsing and real-time information retrieval. He made a tool for chat GPT plus, he made a subscription-based feature in chat GPT, with the help of which you can browse, you can upload files, you can execute codes, Claude used a computer and launched a feature with the help of which he can completely control your computer. So basically, within six months of the function calling, the entire AI world has become a source of tools. And now let's see how this software scenario, where the context was scattered, is now being pan out. So even now, I am the developer, and I was given the same task to implement two-factor authentication. And to do this, I want the help of chat GPT. But now I have all the necessary tools. As soon as a ticket was raised on the jeera, the ticket was raised and assigned to me. So now what I can do is, rather than manually go to the jeera and copy-paste the content of the ticket, I asked the chat GPT directly, to go and check whether I have a new ticket assigned on the jeera or not. And since chat GPT is connected to the jeera through a tool, all this work behind the scenes has been automated. Then I came to know that yes, I have been assigned a new ticket, to develop two-factor authentication. So I suddenly asked the chat GPT, can you fetch the most updated code base from GitHub? So again, if a tool is connected through GitHub, then the whole code base has come. Then I asked the chat GPT, that I would need a schema to make two-factor authentication. Since I

am connected to MySQL with the help of a tool, the scheme has also come to chat GPT. Then I asked security guidelines, whether they are connected to the jeera or not. Finally, I asked the team what they are talking about on Slack, to fetch the information about this particular thing.

Section 9 — 27:57 – 29:26

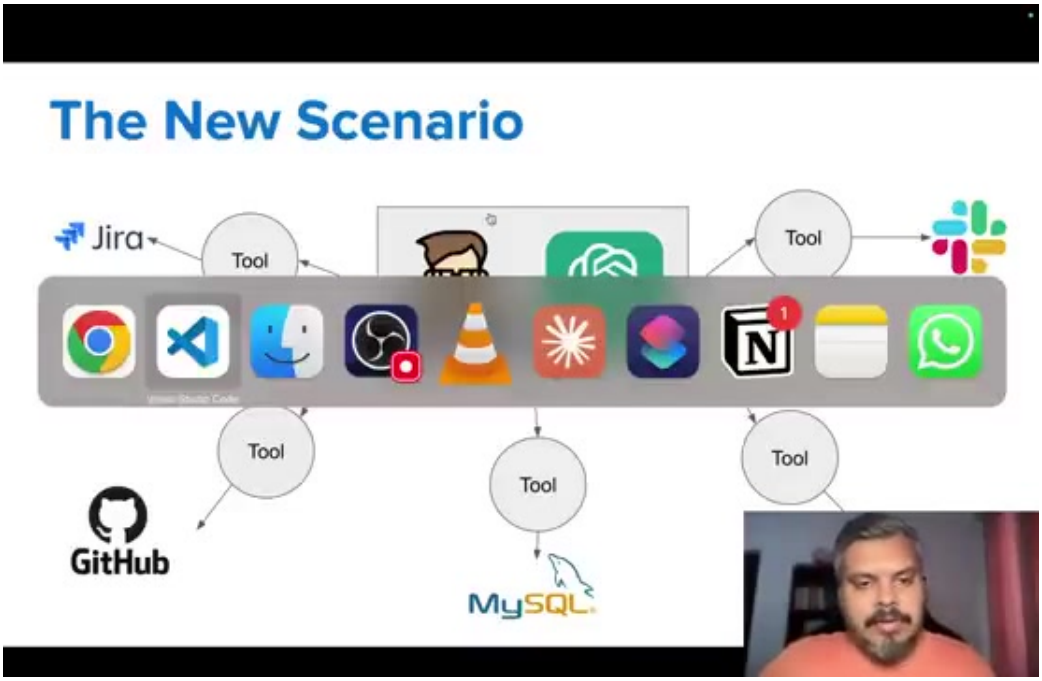

Figure 9. Frame at 29:26.

to fetch the information about this particular thing. Now the entire context has been assembled. Now I asked my question, that now that you can see everything, tell me how can I build a two-factor authentication system? Now you can see the power of tools, that the context that was first scattered, is now connected to our AI chatbot, from chat GPT. And now chat GPT can actually see my entire work. And since it can see my entire work, then obviously, in doing that work, it can help me in a very good way. So in this way, we solved the problem of context assembly, which was stopping us from becoming a full-fledged, unified AI partner, by solving the problem of tools. Now when this function calling slash tools solution came, people felt that they have solved the problem of context assembly. At least for a few days. But after a while, they realized, that even though this tools solution works, and gives you the entire context, but there is a huge flaw in this approach. I will explain the flaw. So as I told you, you have to provide your AI context, the context is scattered across different tools. So what do you do? For every tool, you write a function. For example, let me show you this code.

Section 10 — 29:26 – 29:27

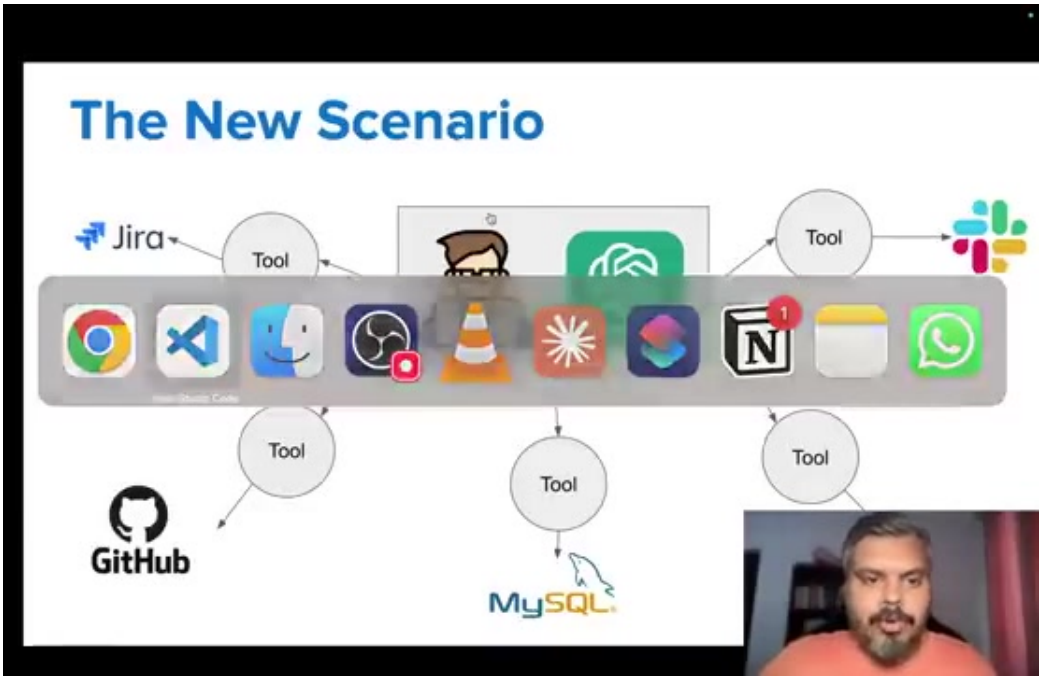

Figure 10. Frame at 29:27.

let me show you this code.

Section 11 — 29:27 – 29:27

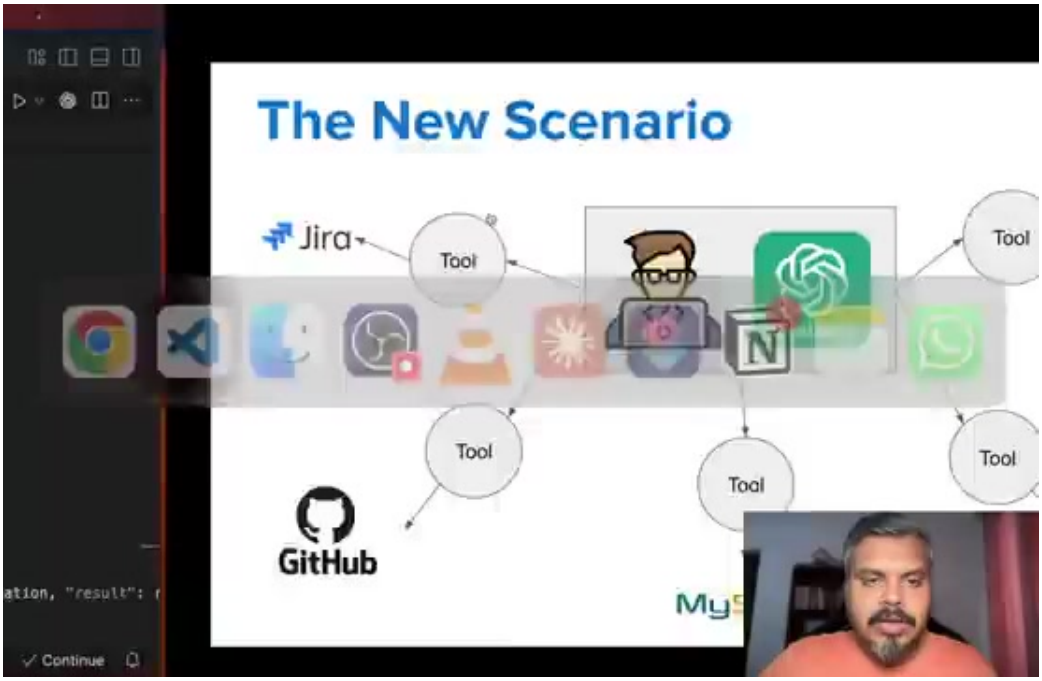

Figure 11. Frame at 29:27.

let me show you this code.

Section 12 — 29:27 – 29:27

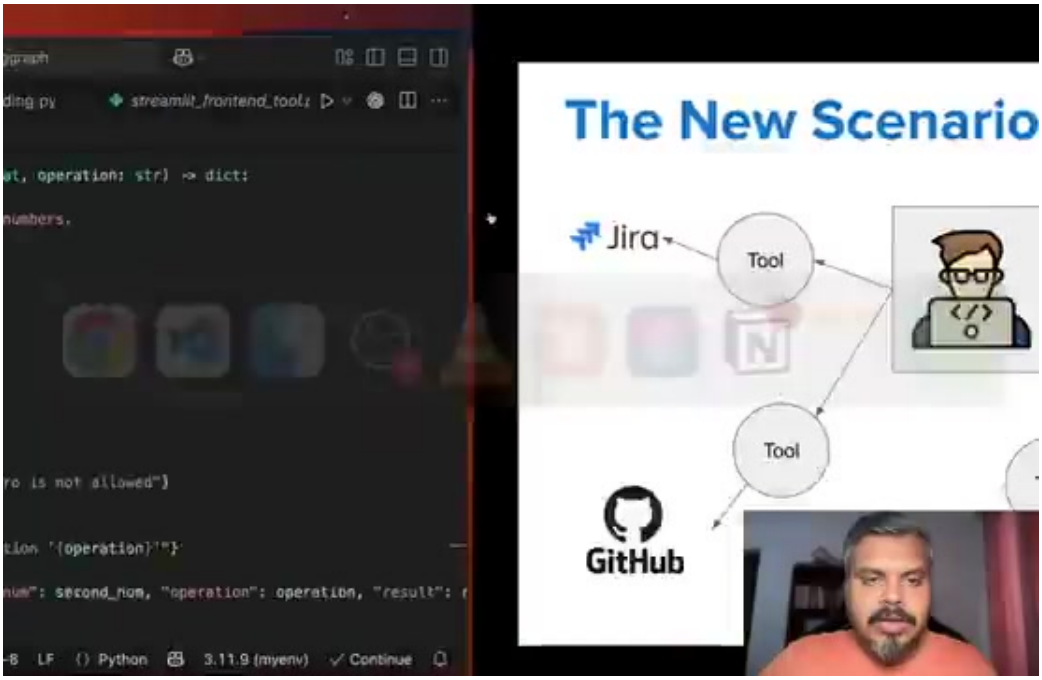

Figure 12. Frame at 29:27.

let me show you this code.

Section 13 — 29:27 – 29:27

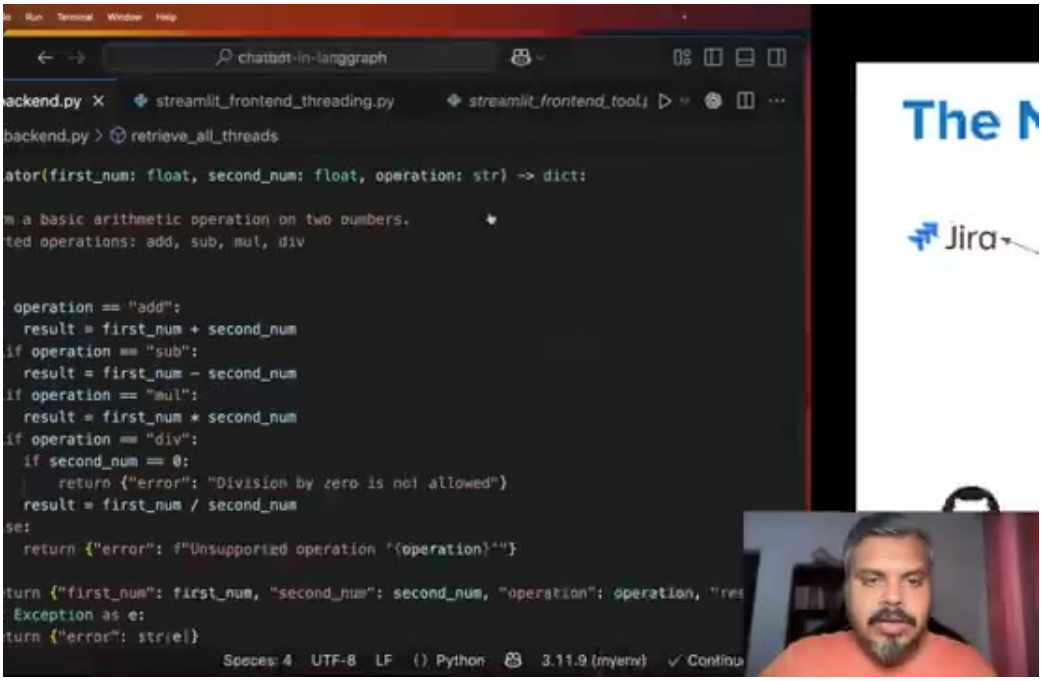

Figure 13. Frame at 29:27.

let me show you this code.

Section 14 — 29:27 – 29:27

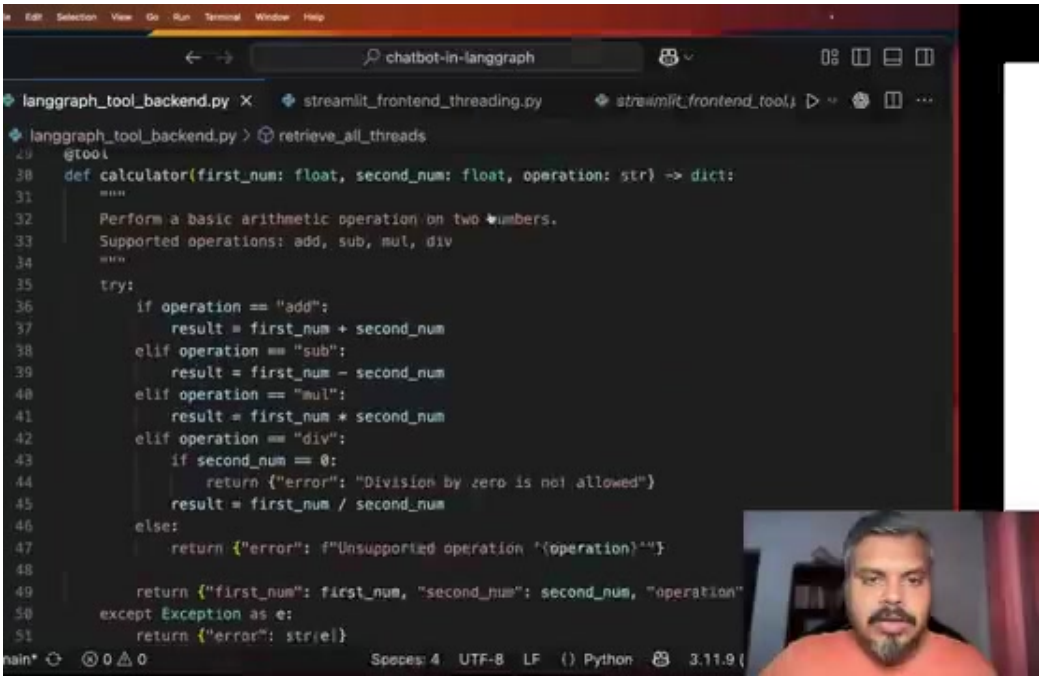

Figure 14. Frame at 29:27.

let me show you this code.

Section 15 — 29:27 – 29:40

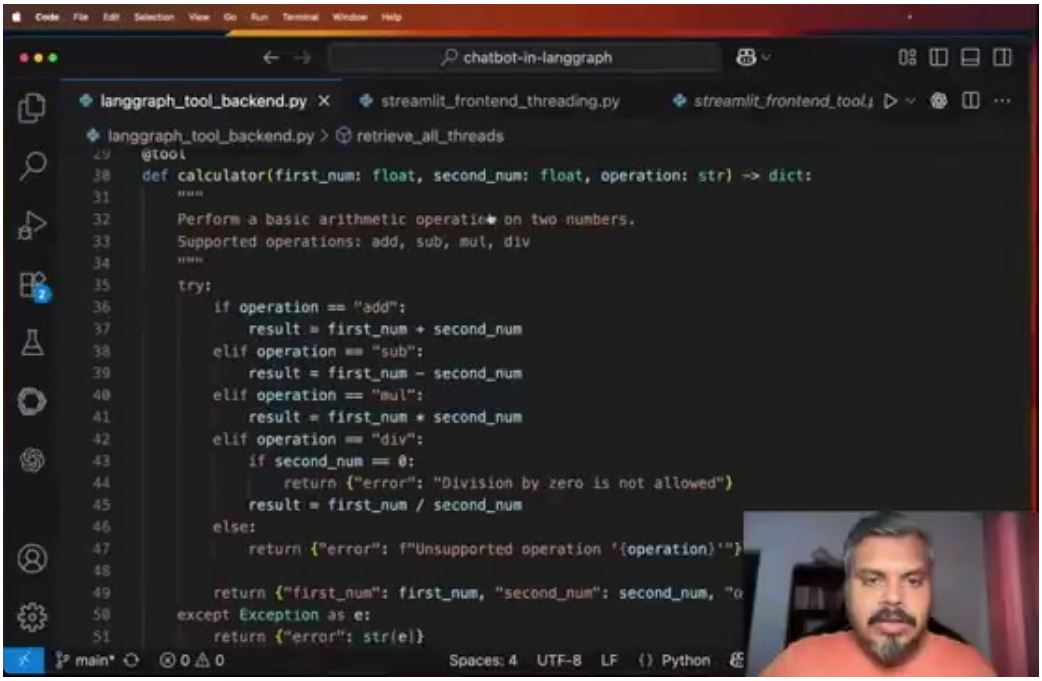

Figure 15. Frame at 29:27.

let me show you this code. This is the calculator we are making in our line graph playlist. I am sorry, the chatbot we are making, is its code. And in that chatbot, we have added two tools. A calculator tool, and a stock market,

Section 16 — 29:40 – 29:53

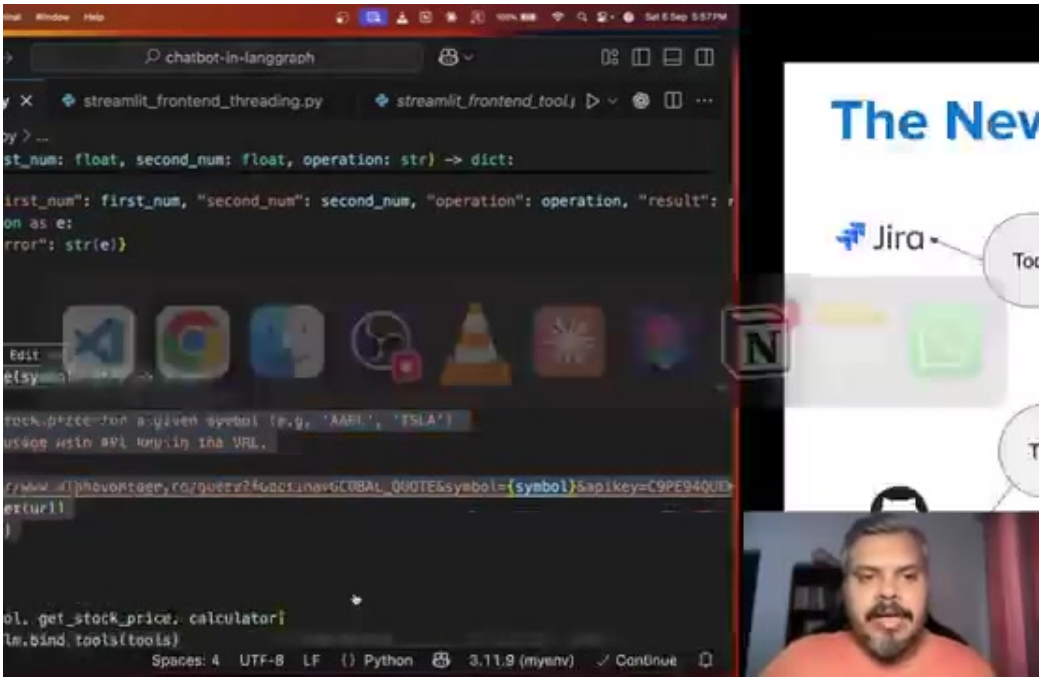

Figure 16. Frame at 29:53.

and a stock market, to fetch any company's stock value. And you see, we have made each function associated with these two tools. This is one function, this is another function. So the basic funda is,

Section 17 — 29:53 – 29:53

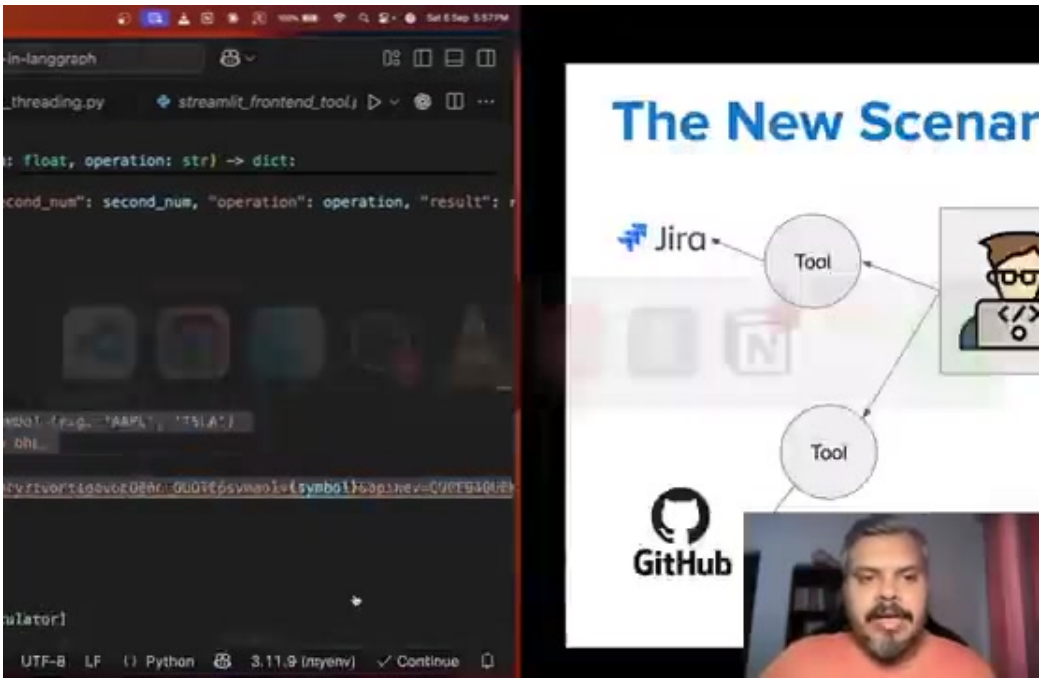

Figure 17. Frame at 29:53.

So the basic funda is,

Section 18 — 29:53 – 30:24

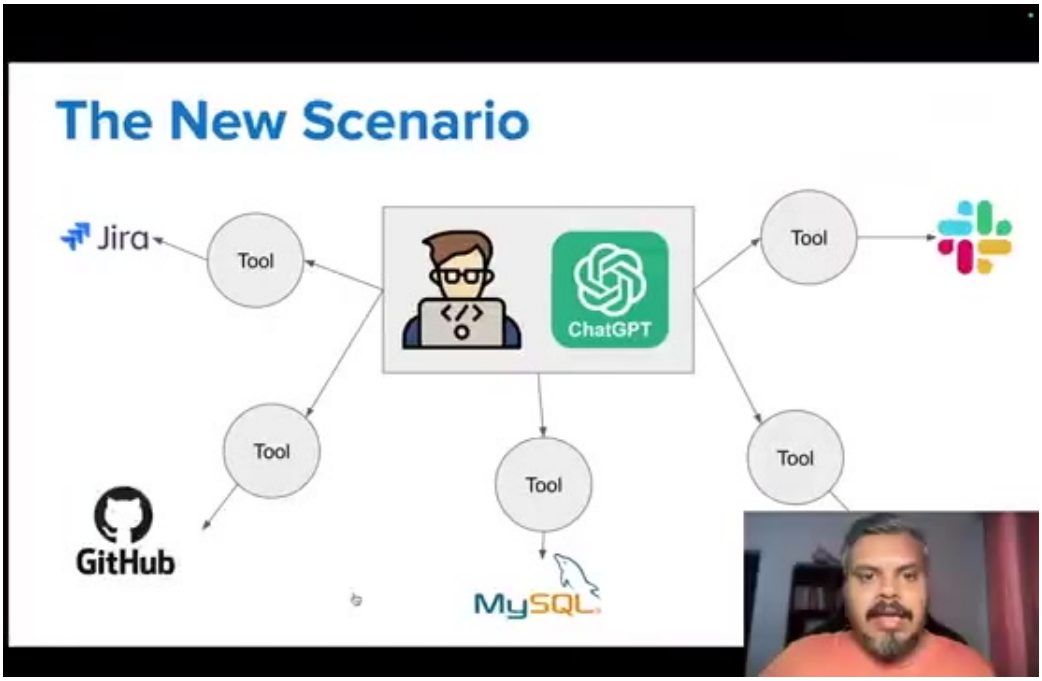

Figure 18. Frame at 29:53.

So the basic funda is, that you have to write a function code for every tool. Now let me tell you how problematic this is. So suppose, a scenario is that, the company we work in, we work with three different AI chatbots. One is a normal chatbot, with which we can ask anything related to the company. The other is a coding agent. So all the developers in our company, software developers, use this coding agent. And there is another chatbot,

Section 19 — 30:24 – 30:56

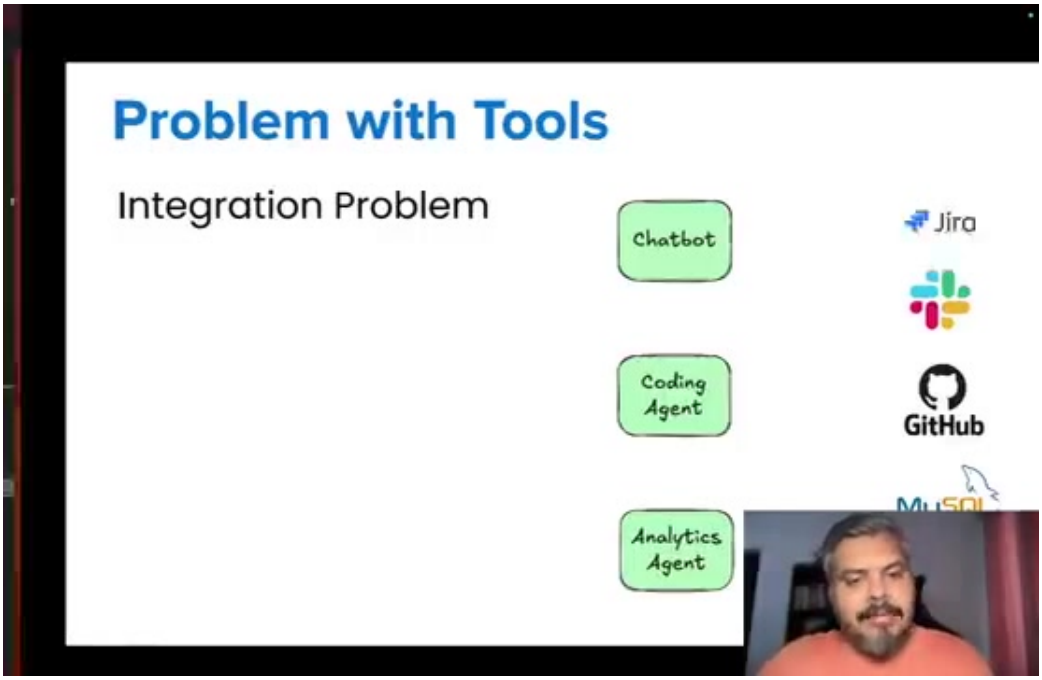

Figure 19. Frame at 30:56.

And there is another chatbot, called analytics agent. So all our data analysts, we give them this chatbot, to do data analysis, automated data analysis. Now for a while, let's assume that our company, works with some 20 different SaaS tools, like jeera, Slack, GitHub, MySQL, Drive, and more like 15 more tools. So now think for yourself, how many functions we will have to write. How many functions we will have to write.

Section 20 — 30:56 – 30:56

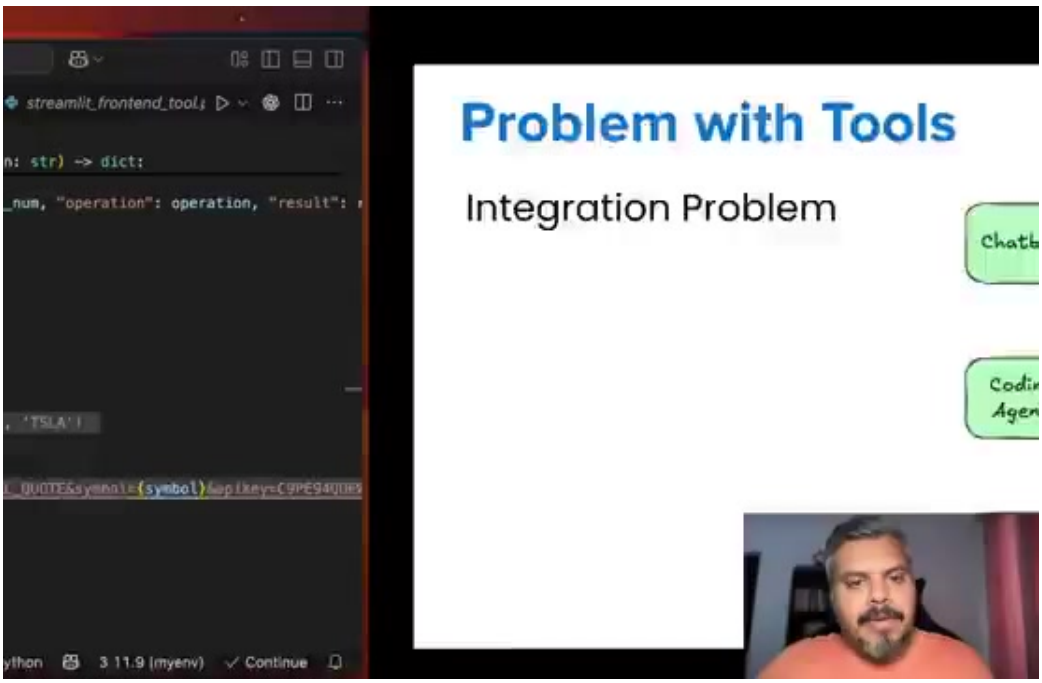

Figure 20. Frame at 30:56.

How many functions we will have to write.

Section 21 — 30:56 – 30:56

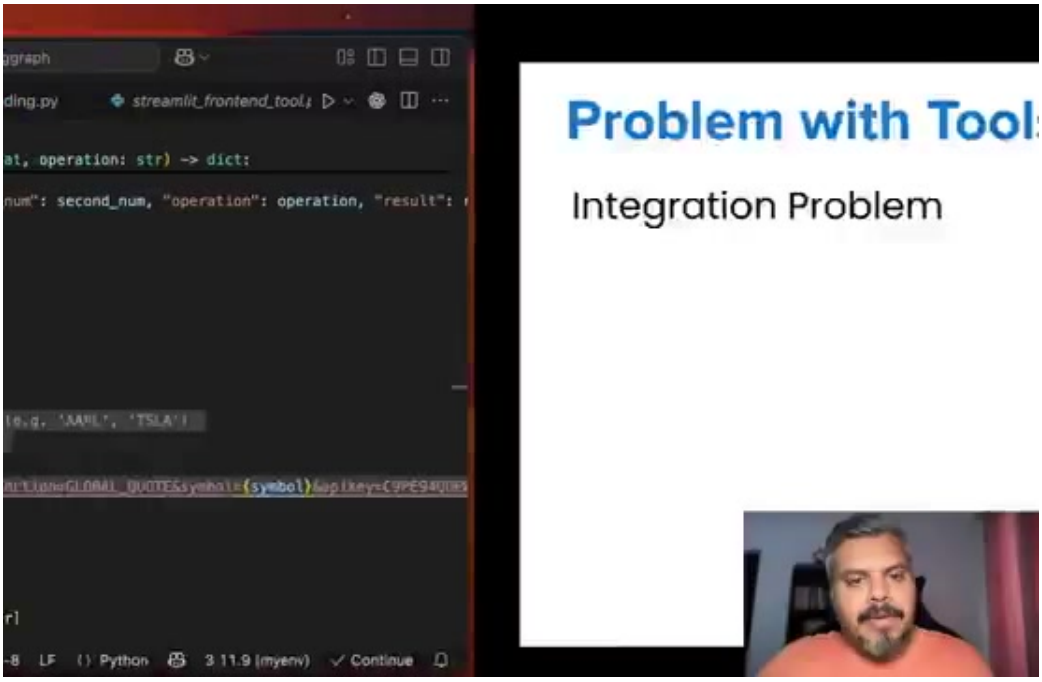

Figure 21. Frame at 30:56.

How many functions we will have to write.

Section 22 — 30:56 – 30:56

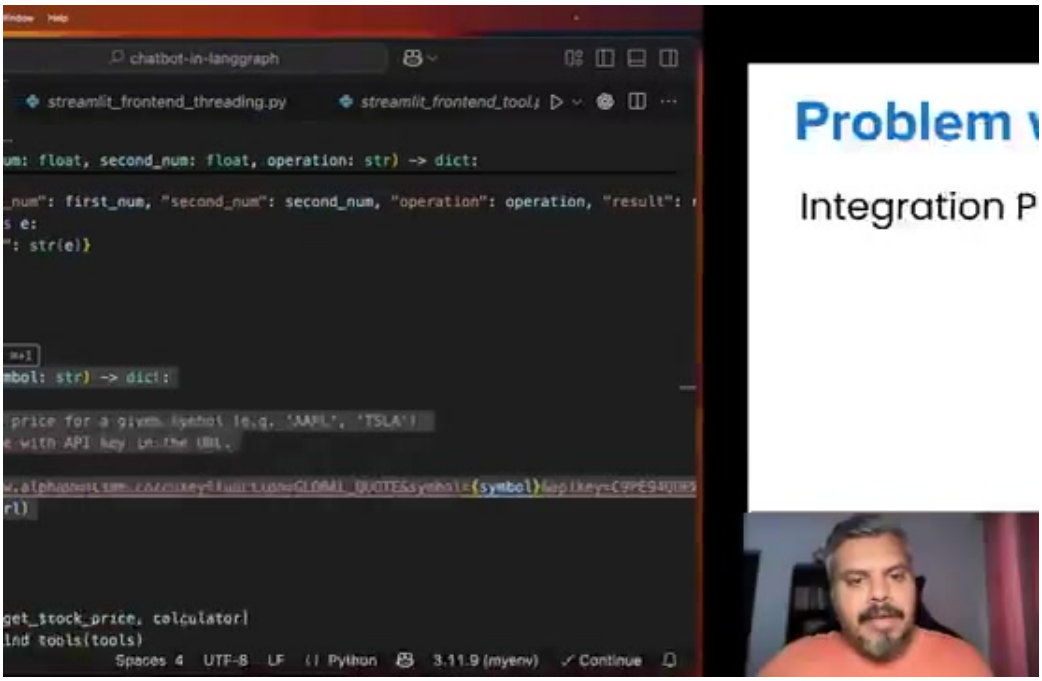

Figure 22. Frame at 30:56.

How many functions we will have to write.

Section 23 — 30:56 – 30:57

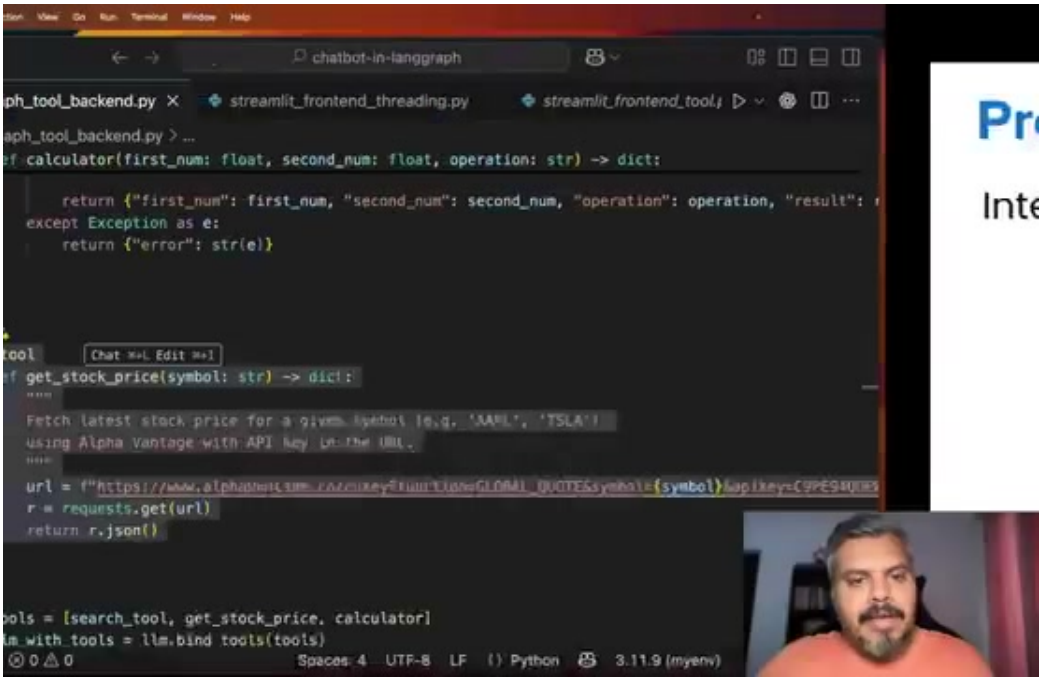

Figure 23. Frame at 30:56.

How many functions we will have to write.

Section 24 — 30:57 – 30:58

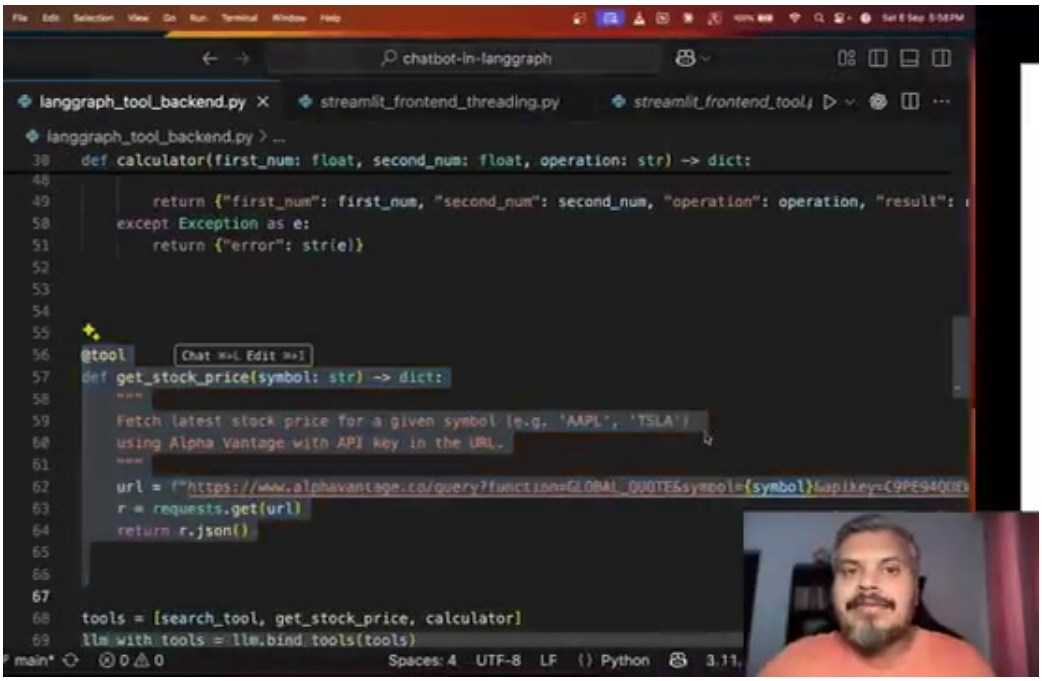

Figure 24. Frame at 30:58.

How many functions we will have to write.

Section 25 — 30:58 – 30:58

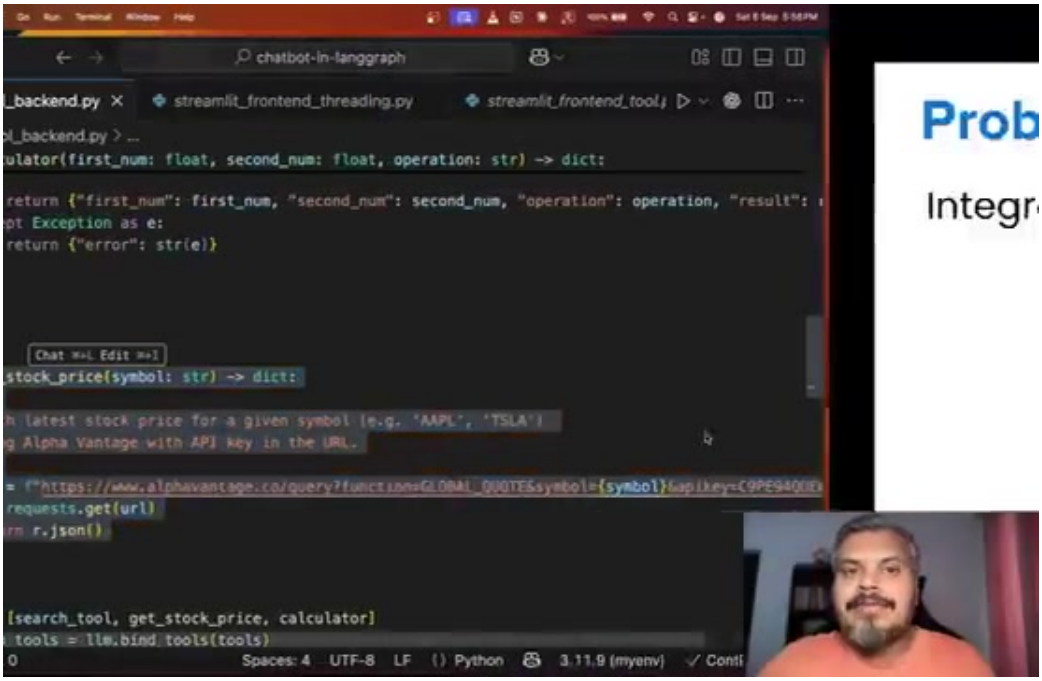

Figure 25. Frame at 30:58.

How many functions we will have to write.

Section 26 — 30:58 – 30:58

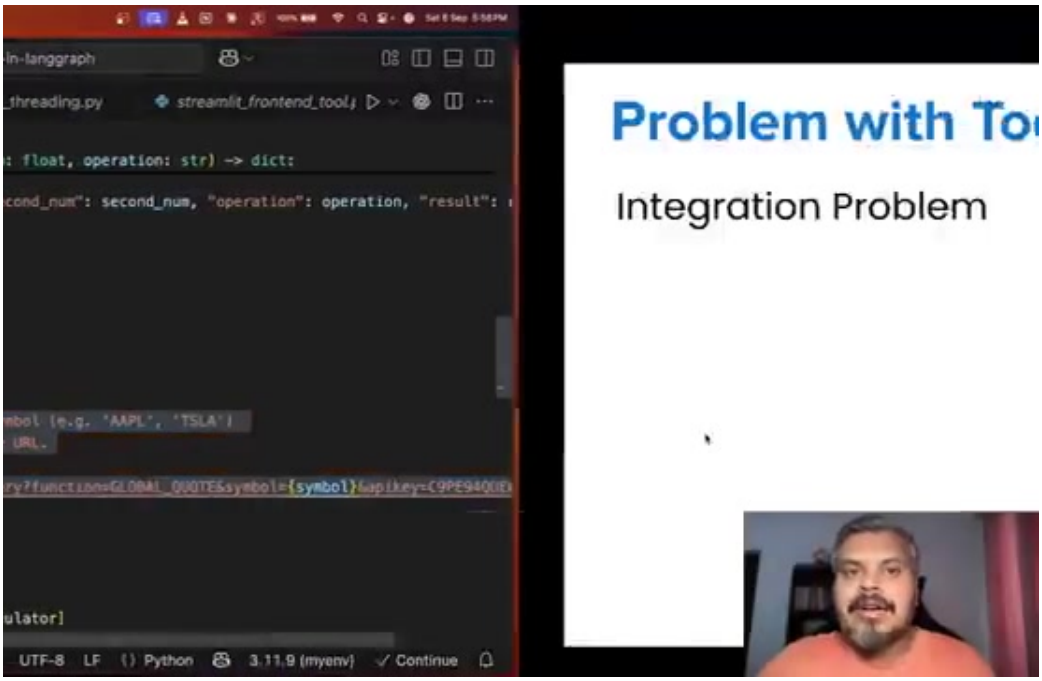

Figure 26. Frame at 30:58.

How many functions we will have to write.

Section 27 — 30:58 – 30:58

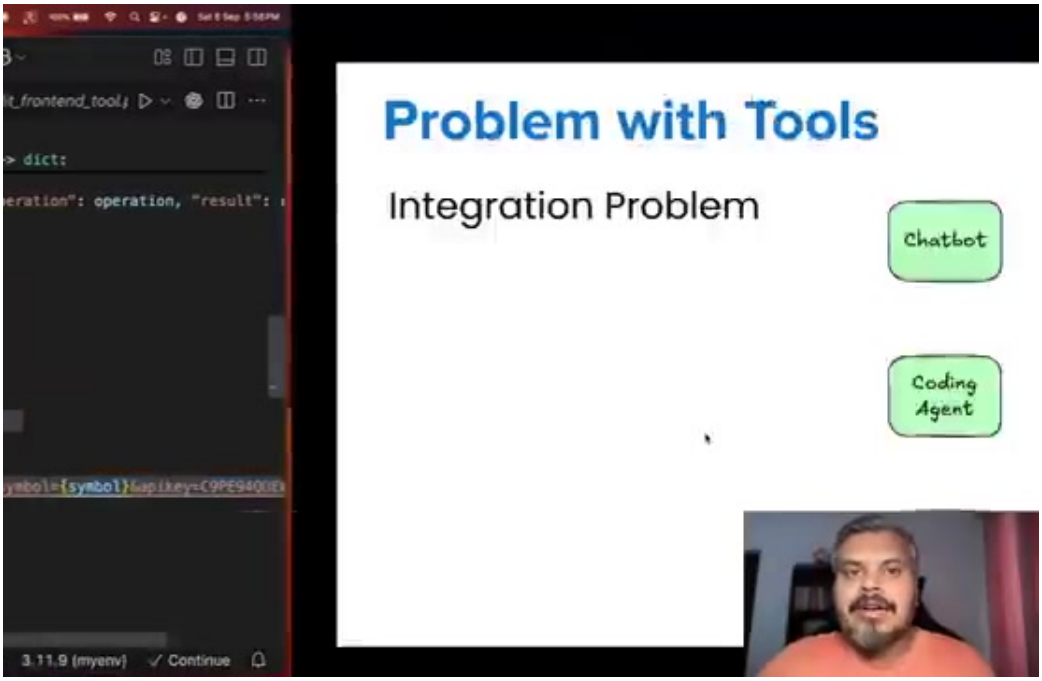

Figure 27. Frame at 30:58.

How many functions we will have to write.

Section 28 — 30:58 – 32:14

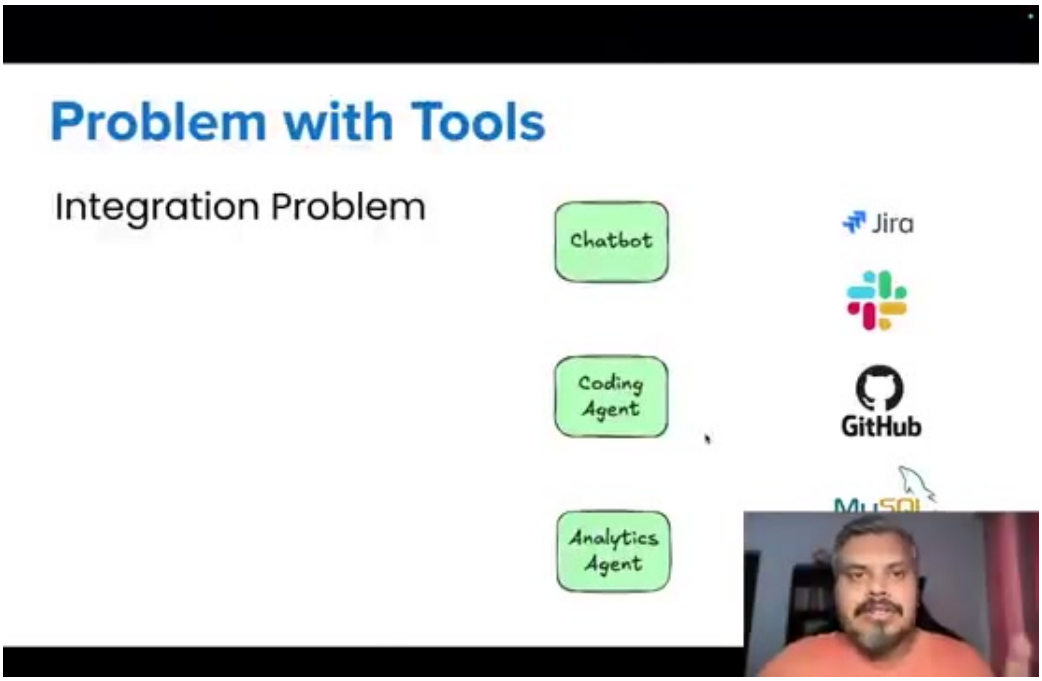

Figure 28. Frame at 30:58.

How many functions we will have to write. I guess you are able to understand, that if you have N AI chatbots, and you have M different tools, or services, then in total, you will have to code N cross M integrations, or functions. Now, this is a small number. In big companies, these numbers can be bigger. Maybe you work with more chatbots, or you work with more SaaS tools, or internal tools. Now making so many functions, writing so many functions, coding so many functions, is a development nightmare. Think about it. Because for every function, you have to write your own authentication method. Every tool will have its own data format, API patterns. Every tool will ask for error handling in a different way. So basically, coding so much, is a different task in itself. You basically have to create a different software team, just to create these integrations. Apart from this, there are more problems. The problem is the maintenance. Suppose you have 3 chatbots, 20 integrations.

Section 29 — 32:14 – 33:31

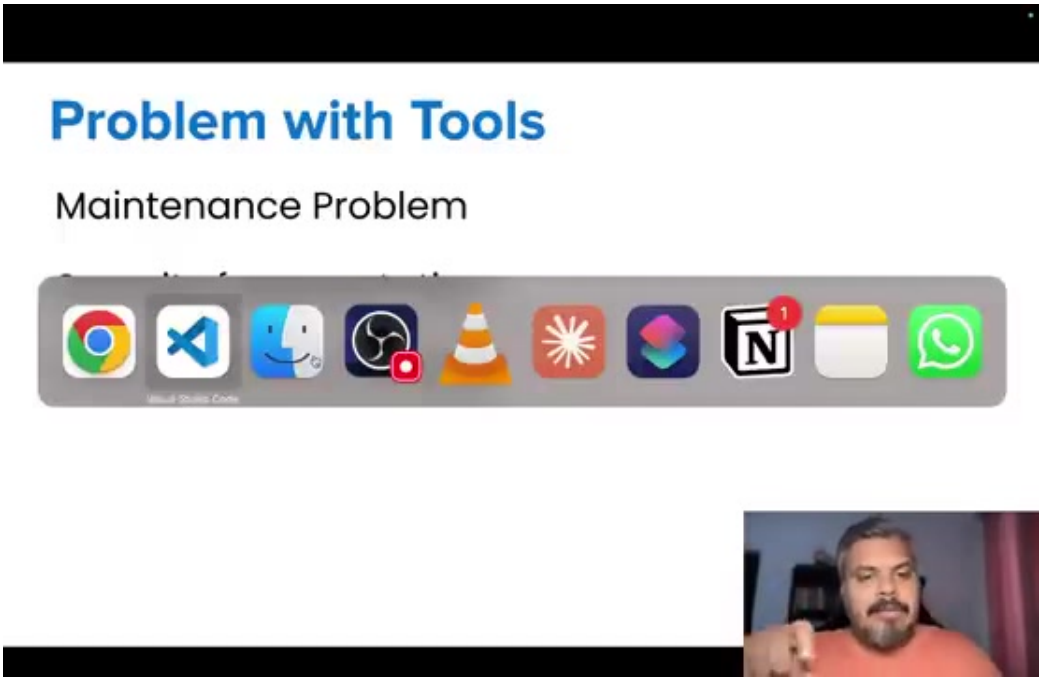

Figure 29. Frame at 33:30.

How many functions do you have to write? 60 functions. Now, you have to maintain these 60 functions. If Google Drive has changed it in its API, then automatically, all the integrations of Google Drive have failed. You are not able to talk to Google Drive in 3 or 4 chatbots. So again, you have to debug it. Apart from this, there is a security problem. When you work in big companies, you make sure that your software works securely. But since you have written 60 different integrations, and you have your own OAuth, your APIs keys, you cannot manage all of them in a single place. If you have all the information in different files, then your security is fragmented. And it is possible that some kind of hack will be performed there. And the fourth problem is, that in this whole approach, your cost and time is getting wasted. Think about how time is getting wasted. If I have a chatbot, and I have to connect 20 tools with it, then I have to write individual integrations for 20 tools. I basically have to make 20 functions like this,

Section 30 — 33:31 – 33:31

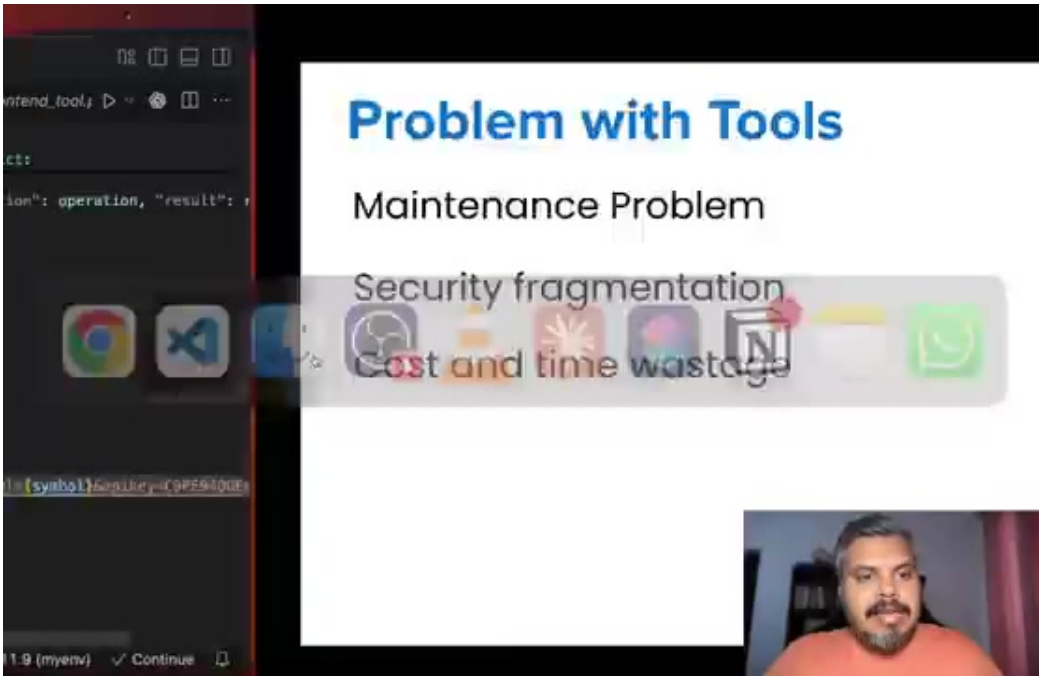

Figure 30. Frame at 33:31.

I basically have to make 20 functions like this,

Section 31 — 33:31 – 33:32

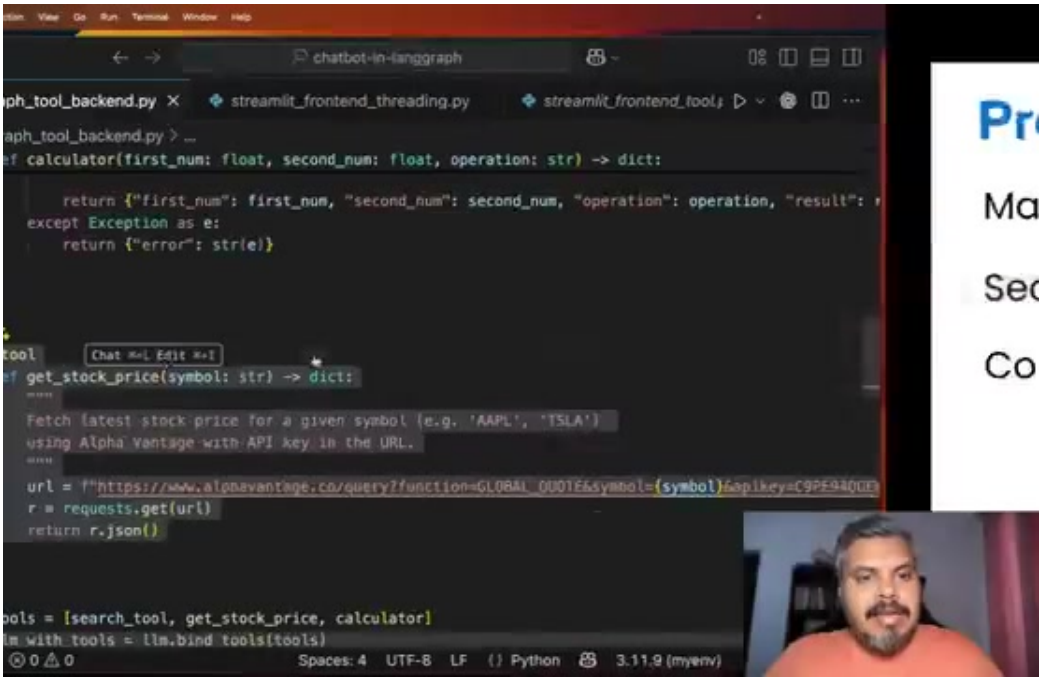

Figure 31. Frame at 33:31.

I basically have to make 20 functions like this,

Section 32 — 33:32 – 33:33

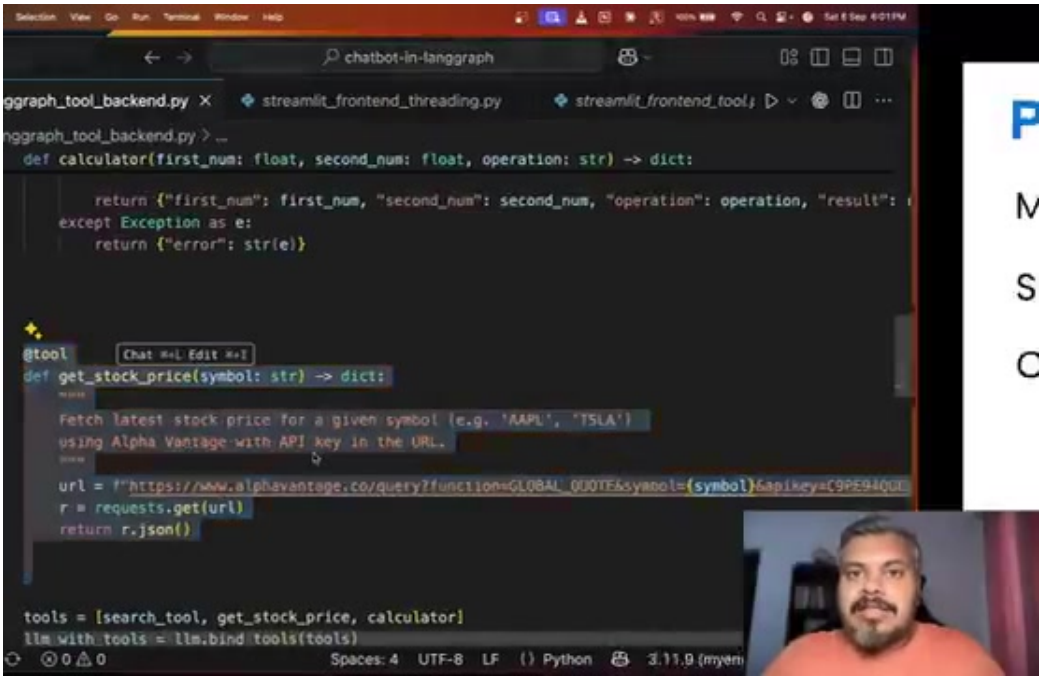

Figure 32. Frame at 33:33.

I basically have to make 20 functions like this,

Section 33 — 33:33 – 33:33

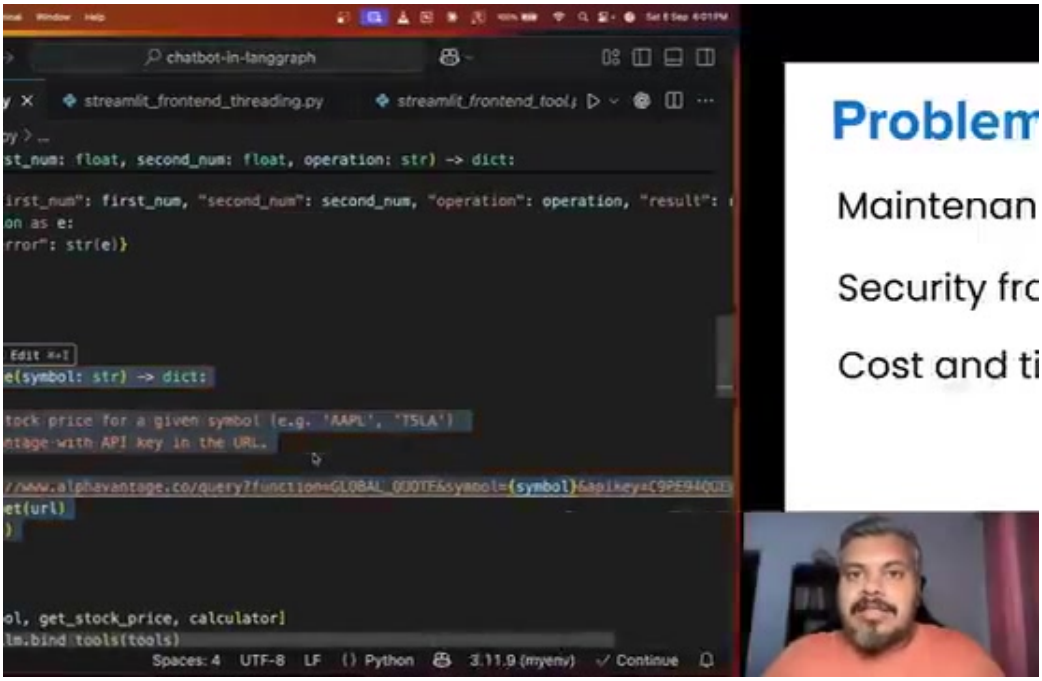

Figure 33. Frame at 33:33.

I basically have to make 20 functions like this,

Section 34 — 33:33 – 33:33

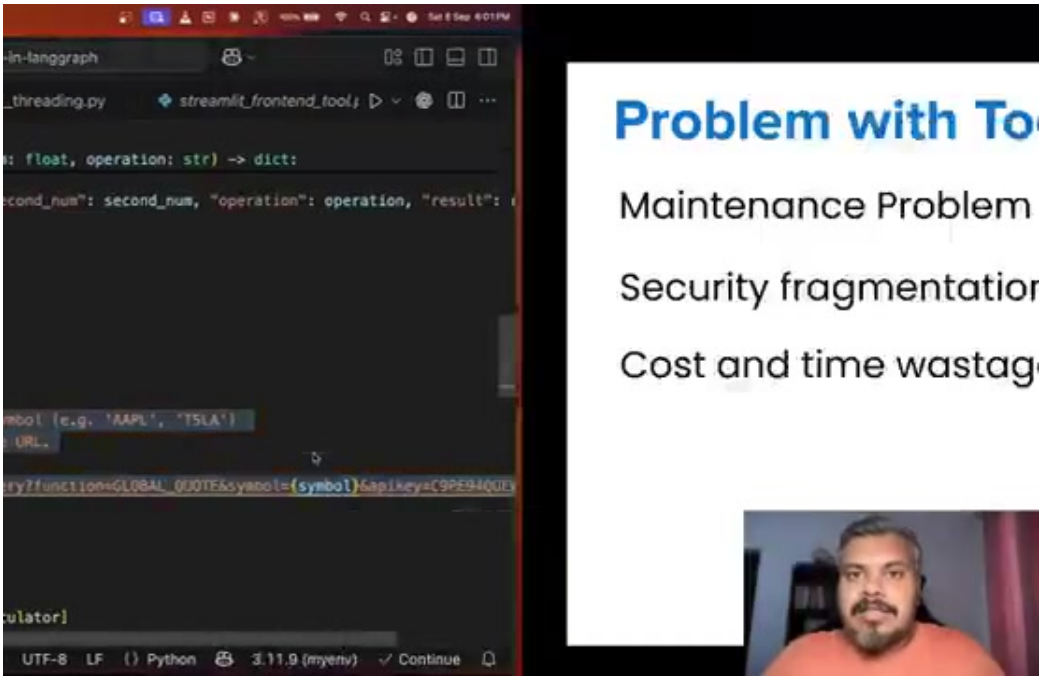

Figure 34. Frame at 33:33.

I basically have to make 20 functions like this,

Section 35 — 33:33 – 33:33

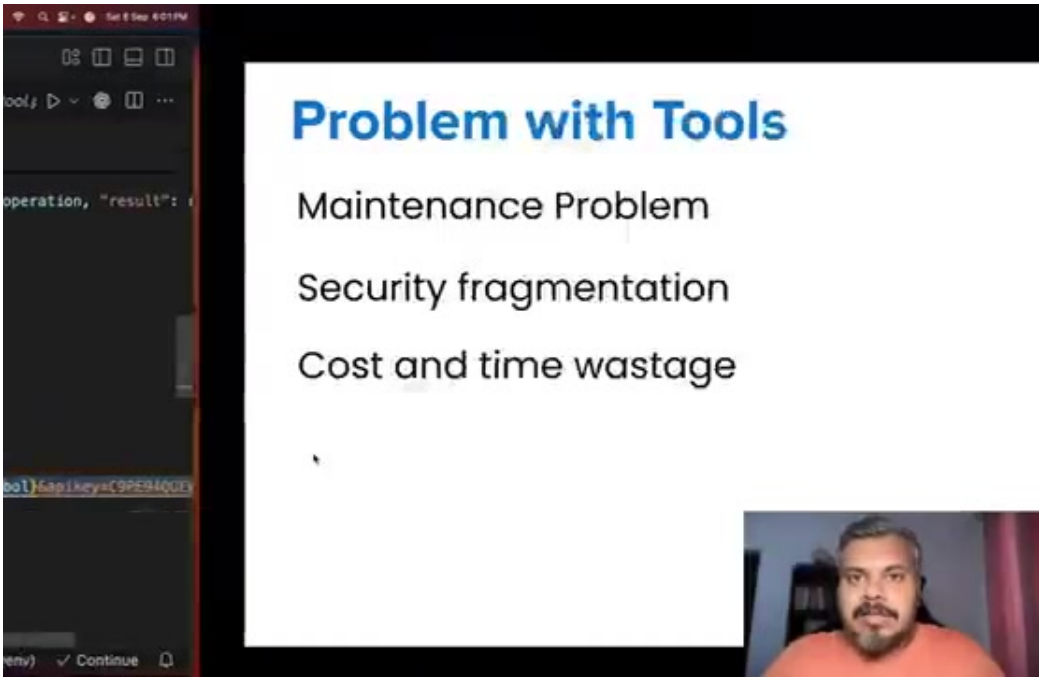

Figure 35. Frame at 33:33.

I basically have to make 20 functions like this,

Section 36 — 33:33 – 39:44

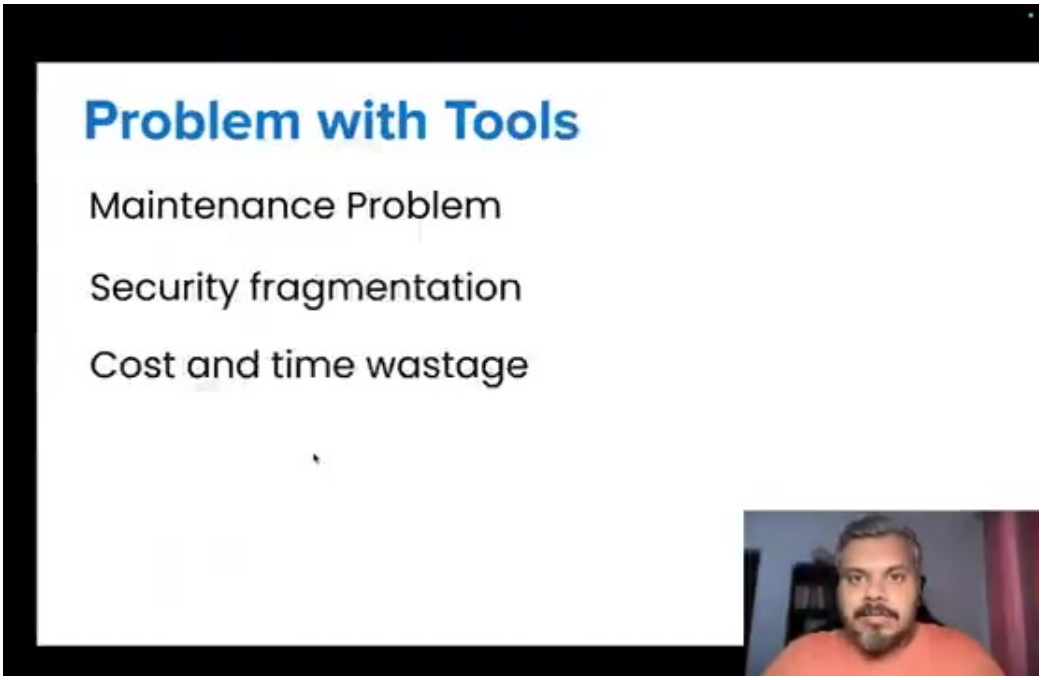

Figure 36. Frame at 33:33.

which is very complex in itself. So every integration will take time to code every function. So my chatbot should be fully functional. It may take 2-3 months. Right? After that, cost is involved. Basically, I have to set up a whole team that handles the entire integration part. Now I have to see their salaries. Now think about my end goal. Why am I doing this? So that my existing developers can do their job easily. Now that their job is easy, I have to hire other developers to do this job. Just see the irony in this. You went to do something easy, but it became more difficult. So what I am trying to say is, that the solution of tools and function calling, that we implemented in order to solve the context assembly problem, actually brought an integration problem in itself at scale, which again seemed very heavy for people to solve. Now let's summarize the integration problem of this function calling slash tools. The basic overview of this problem is that our company is using more than one AI chatbots. It's possible that our company is using perplexity, along with cursor, and also using chat gpt. And we want these three chatbots to connect with GitHub. Now the problem is that to do this job, we have to make three different integrations. One for perplexity and GitHub, one for cursor and GitHub, and one for chat gpt and GitHub. And the problem is that these three integrations will look different from each other. So the problem is that every AI tool is building its own way to call every API. It would be nice if we somehow make sure that rather than making three integrations, we would just make one single integration of GitHub. And the same integration would work with perplexity, the same with cursor, the same with chat gpt. This is what we want to achieve. And this is how we can solve this integration problem. And then who came to solve this problem? MCP, Model Context Protocol. So guys, let's talk about MCP. Let me tell you in simple words how MCP works. So there are two entities in MCP. One is client and the other is server. And the whole communication is going on between these two entities. Now don't start thinking about what is the new thing by calling client or server. Actually, the client in MCP is your AI chatbot. Basically your chat gpt, cursor, perplexity, or your own AI chatbot. We are calling that client. And the server is basically that service through which you want to connect your AI chatbot. Like GitHub, Google Drive, Slack. Now the general architecture of any MCP application shows that you have a single client and he is connected to a lot of servers. He is connected to a lot of tools. Meaning an AI chatbot is connected to multiple tools. So one tool is GitHub, one is Google Drive, one is

Slack. Now the whole communication is going on, the whole conversation between the client and the servers. The language we are talking about is called MCP or Model Context Protocol. Okay, so I really hope you understand what I have just said. Now I know that you must be curious how I can make my AI chatbot MCP client and how I can make my tools MCP server. Now the answer is that if you read the MCP protocol properly, its documentation properly, then you can write the entire code on your own from scratch which will establish this MCP communication between client and server. But Anthropic gives you a Bani Bani library or SDK. So if you want to make your AI chatbot MCP compatible client, then you don't have to do anything, you have to install an MCP client SDK on your machine. where your AI chatbot is running. And if you want to make your tools MCP compliant server, then again you have a library called MCP server SDK, you will use it to build your tools. This is the biggest difference. This is how you are becoming MCP client, this is how you are becoming MCP server. Now let's go to the technical level and understand how MCP is different from function or tool calling, which we read just before. So in function or tool calling what you do is, you write a function to access the API of the tool you want to connect with. In your AI chatbot, which I showed you earlier, we made two different tools in our chatbot, calculator one and stocks one. So what happens is that even here a client server model is being followed. For example, if we want to connect our AI chatbot with a weather tool,

Section 37 — 39:44 – 45:57

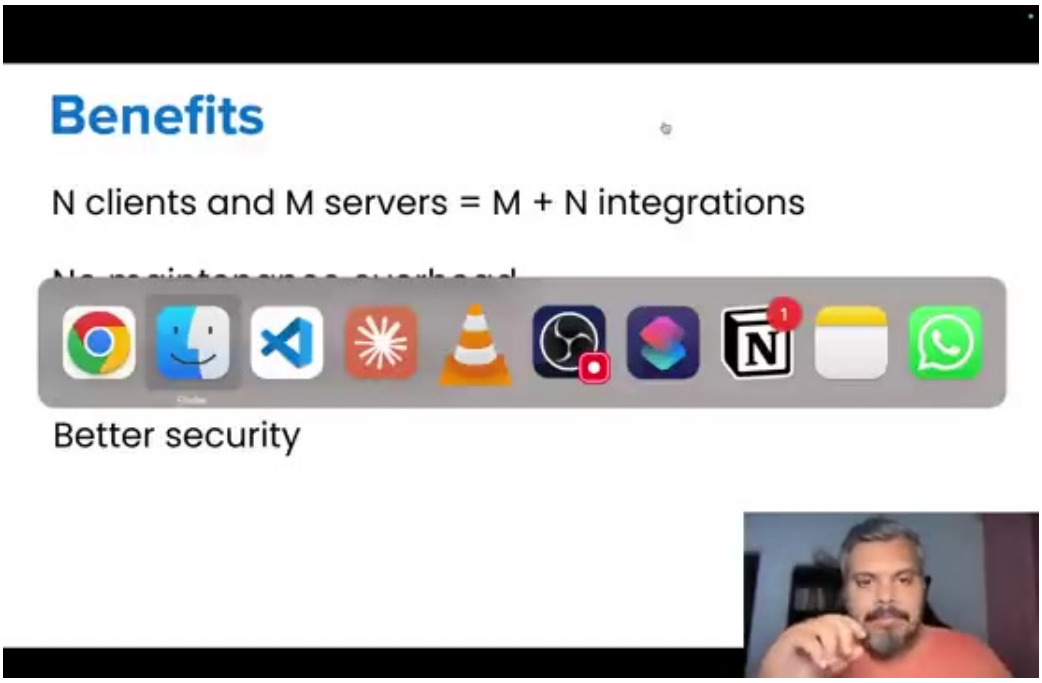

Figure 37. Frame at 45:56.

For example, if we want to connect our AI chatbot with a weather tool, then obviously somewhere on the server there will be an API of the weather tool, which is written in fast API and will look something like this. So this kind of our server is done. And then to access it what we do is, we write a function. In our AI chatbot code. And here we write a code to get data by hitting this API. So this kind of our client side is done. So this is normal function or tool calling. Now let me tell you what happens in MCP. So there will be a server in MCP, where the weather data will be recorded. The server here will be made with the help of the MCP library. The code is different, but internally it is using the same API. The main difference comes

from the client side. On the client side, you don't need to write any code in your AI chatbot. Since you have already integrated the client and server and configured it, and they are speaking the same language that is model context protocol, then you don't need to write a code and fetch the API data separately. All this work is done by the server. So let me explain you again on a technical level. In function calling, there is a server code that is written by the company, and a client code that is written by you. And these two codes work together, then the task is completed. In MCP, all the heavy lifting is done by the server. MCP server. And you don't have to do anything on the client side. You just have to connect the client and server. Once both are connected and the same language is spoken, then our client knows, our AI tool knows that it has access to this server, and it can fetch the tool of your choice from that server. So this is the major difference between MCP and function slash tool calling, that in MCP, the server does the heavy lifting on the client side, on the AI chartboard side, you don't have to do anything. You just have to connect and sit. So suppose if you are making a GitHub server in MCP, then it is the responsibility to do everything on the GitHub server. The whole business logic will be implemented on this server. The work of authentication will be implemented on the server. The rate limiting of API will be seen on the server. Data format, translation, LLM, everything will be seen on the server. The code for error handling will also be written on the server. And the right error code will be sent to the client, if something goes wrong. And what you have to do with your client, you just have to connect with this server. And you have to call it the same language, MCP. This is the biggest difference between function slash tool calling and MCP. I really hope you understand a little bit. In future videos, we will study it in a lot of detail. But right now I want to give you an overview, a gist. Now let's discuss once, what is the benefit of implementing this particular logic? So as I said, the biggest benefit of MCP is that you don't have to do anything on the client side. You are doing all the heavy lifting servers. So after MCP came, this happened. All the famous service providers, Github, Google Drive, Slack, all of them made their own official MCP servers. Which basically means that any MCP compliant AI tool can easily connect with these servers. Now the benefit of this is that your integration problem, suppose there are 3-4 boards in your company and you want to connect with 10 services. So you had to write 3 multiplied by 10, 30 unique integrations. Now you don't have to do this. Now if you want to connect with 10 MCP servers, then you only need 10 integrations. And those integrations are also written on the server side. The service provider is writing it himself. You don't need to write any integration on the client side. You can just configure your AI tool to connect with the server. So first where there were m cross n integrations, now in return there are only m plus n integrations. And the actual code written on the server side is delegated. You don't have to do anything to the client. This is the first benefit. Second benefit is there is no maintenance overhead. Since I am not writing any code on my site to connect, then nothing can go wrong there. If the API is updated tomorrow, then it is the server's headache. The server will update its code. I don't need to do anything on my site. On the client side there will be no changes. Third, reduced cost and time. Suppose I have to connect my AI chatbot with 10 different services. So first I had to make 10 integrations in which I used to take time. But what I have to do in today's date? I just have to make AI chatbots. And 10 servers are already available. I will directly connect them on day 1. So on day 1 I will get 10 servers easily. So there is time left here. And obviously cost is left because I don't have to hire different engineers to create and maintain these integrations. And the last point is that we get better security. Why? Because when you connect an AI chatbot to multiple services,

Section 38 — 45:57 – 45:58

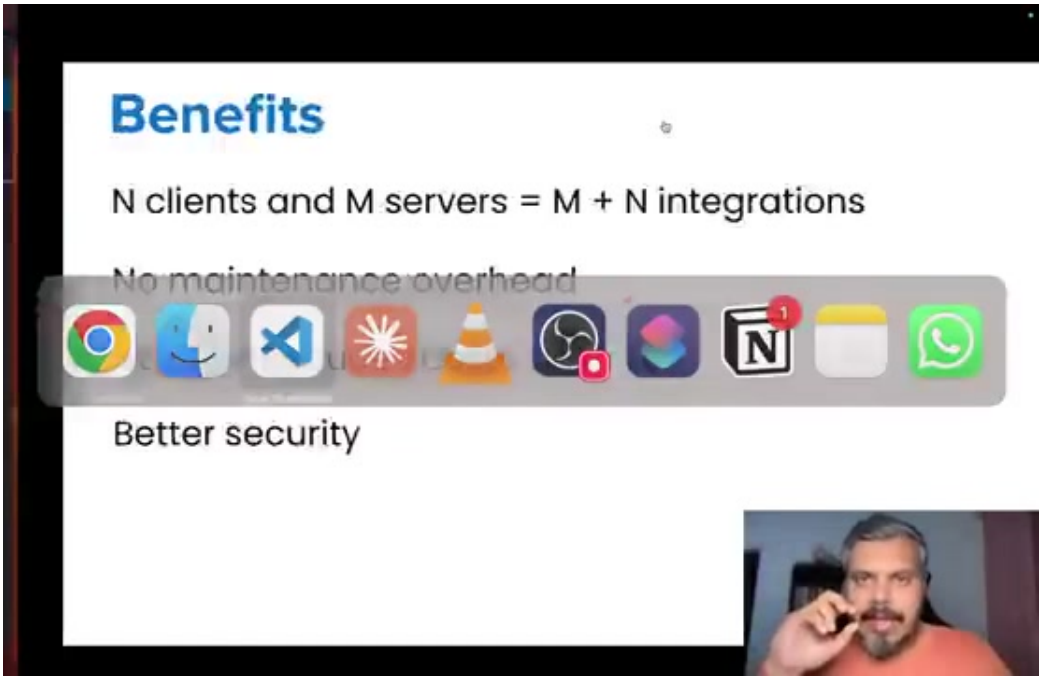

Figure 38. Frame at 45:58.

Why? Because when you connect an AI chatbot to multiple services,

Section 39 — 45:58 – 45:58

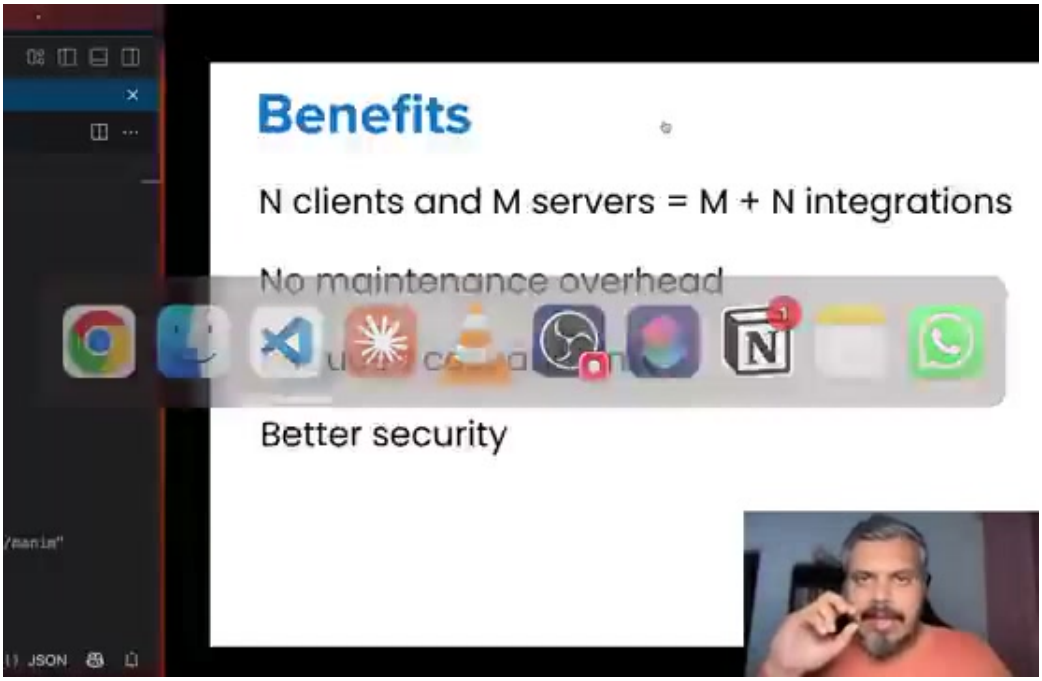

Figure 39. Frame at 45:58.

Why? Because when you connect an AI chatbot to multiple services,

Section 40 — 45:58 – 45:58

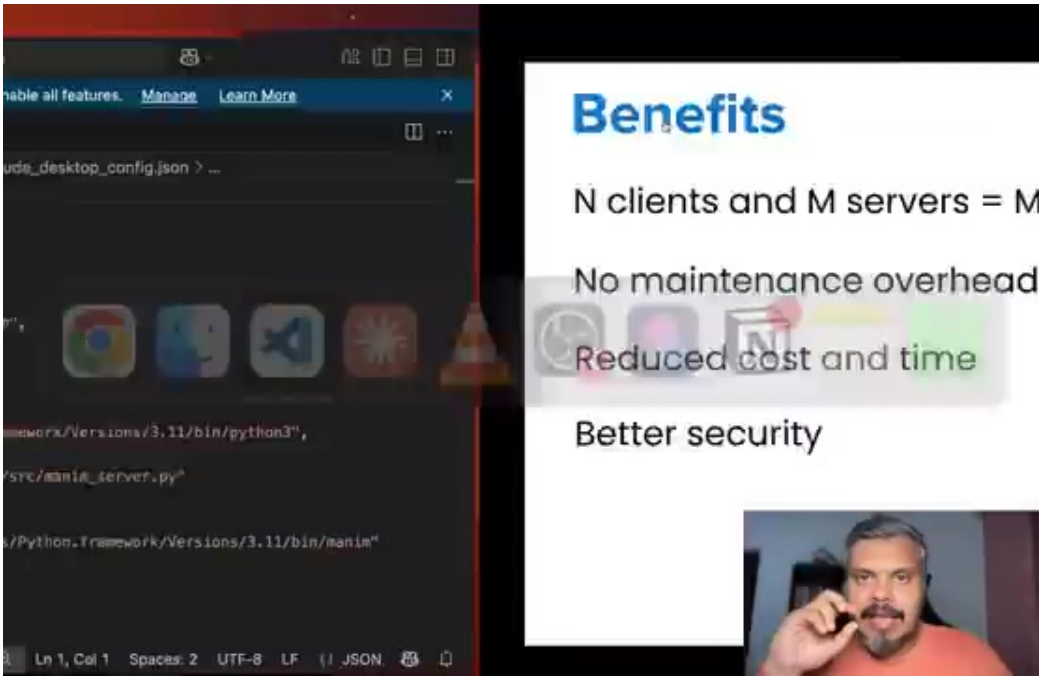

Figure 40. Frame at 45:58.

Why? Because when you connect an AI chatbot to multiple services,

Section 41 — 45:58 – 45:58

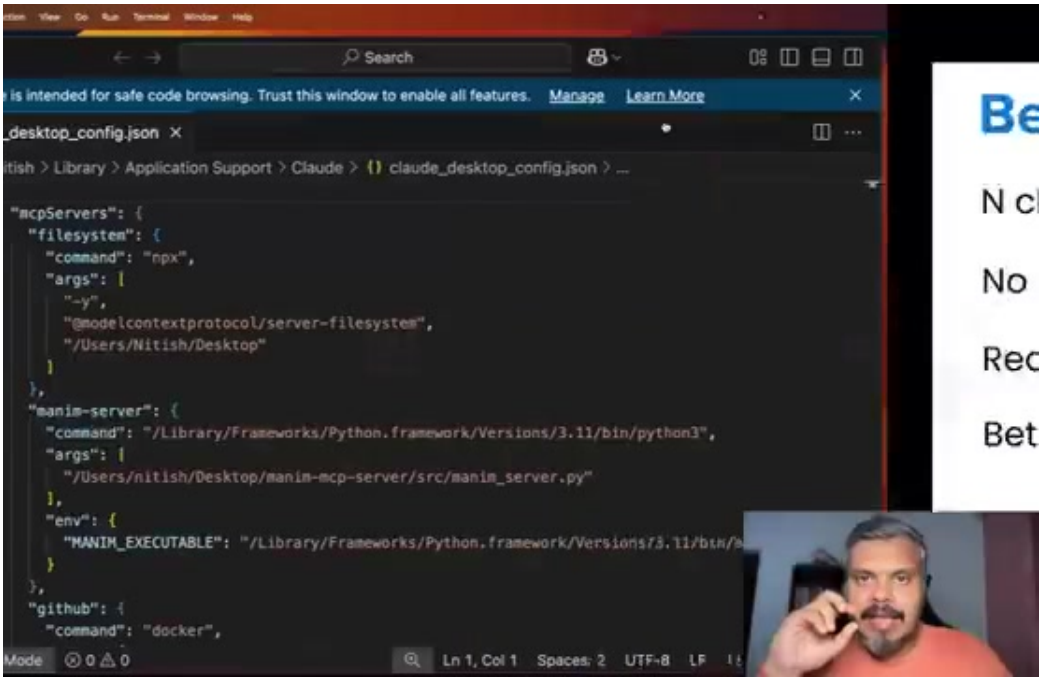

Figure 41. Frame at 45:58.

Why? Because when you connect an AI chatbot to multiple services,

Section 42 — 45:58 – 46:28

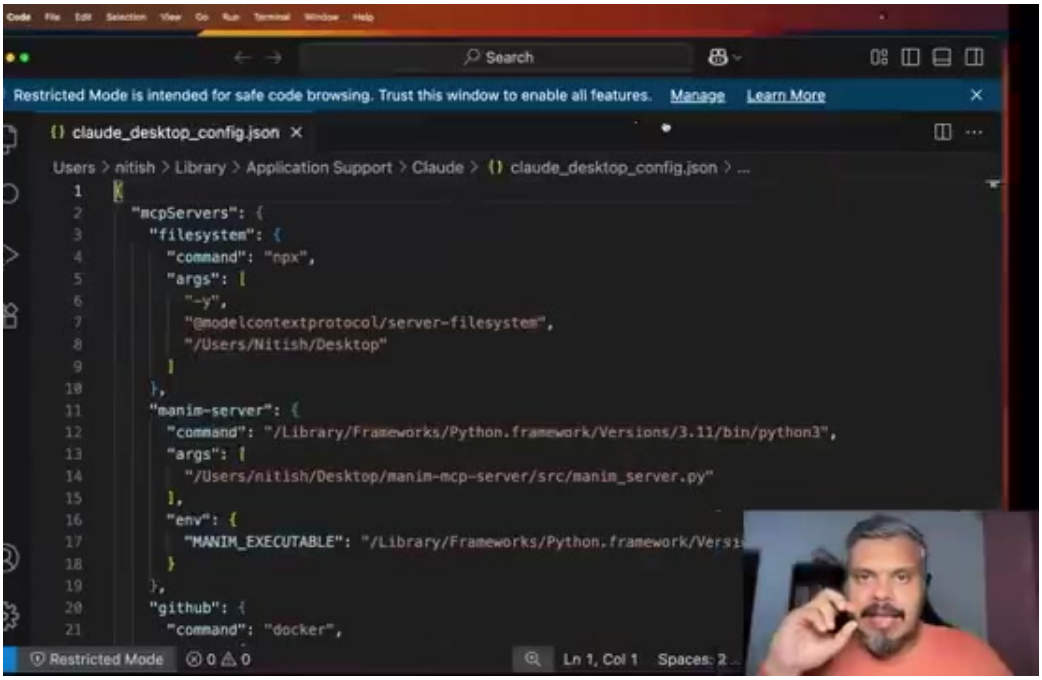

Figure 42. Frame at 45:58.

Why? Because when you connect an AI chatbot to multiple services, then you simply have to maintain a JSON file, a config file where all your connections are written in one place. For example, this much part that you are showing is the code to connect my AI chatbot with GitHub. This is it. I just need this much code. Here I have given my personal access token and this is connected to my GitHub account.

Section 43 — 46:28 – 46:58

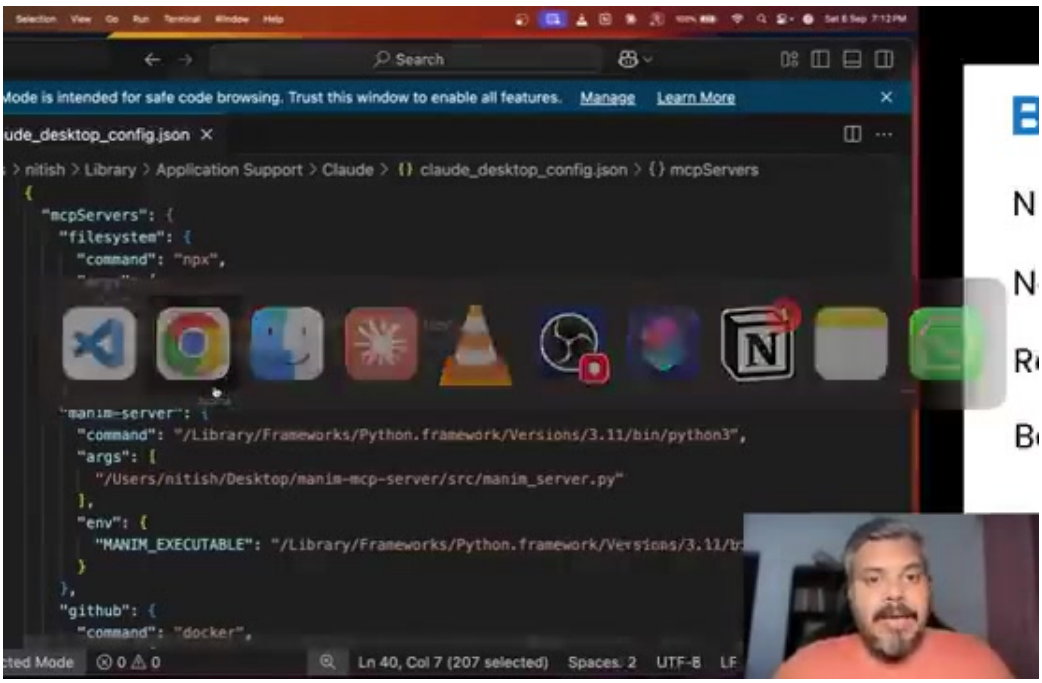

Figure 43. Frame at 46:58.

Here I have given my personal access token and this is connected to my GitHub account. Similarly, this much code is connected to my AI chatbot with Notion. Again I have given my API key. Now managing or auditing this single file is much simpler than the old case where we had kept 10 different APIs in different files and tools. So this is a much better security model as well. So these are the main benefits of MCP.

Section 44 — 46:58 – 46:58

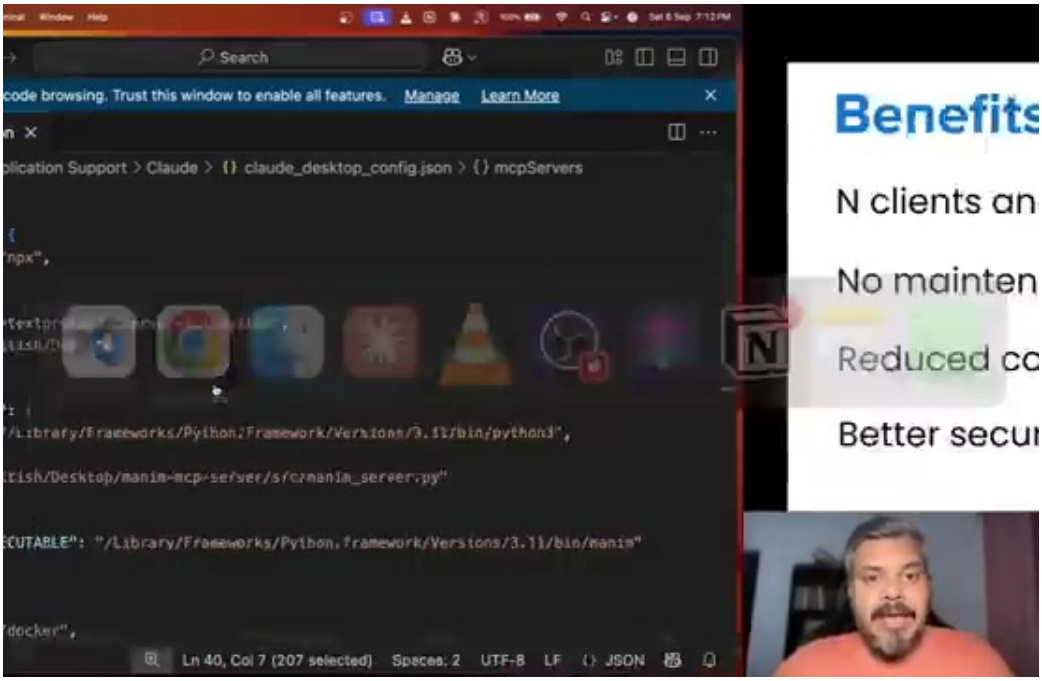

Figure 44. Frame at 46:58.

So these are the main benefits of MCP.

Section 45 — 46:58 – 46:58

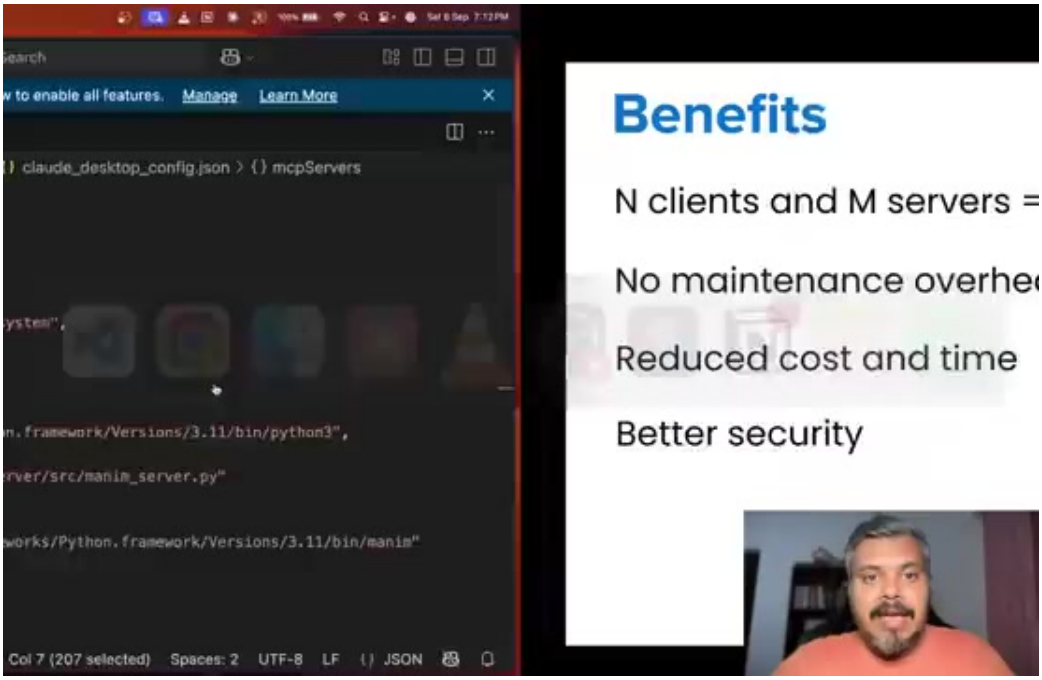

Figure 45. Frame at 46:58.

So these are the main benefits of MCP.

Section 46 — 46:58 – 47:08

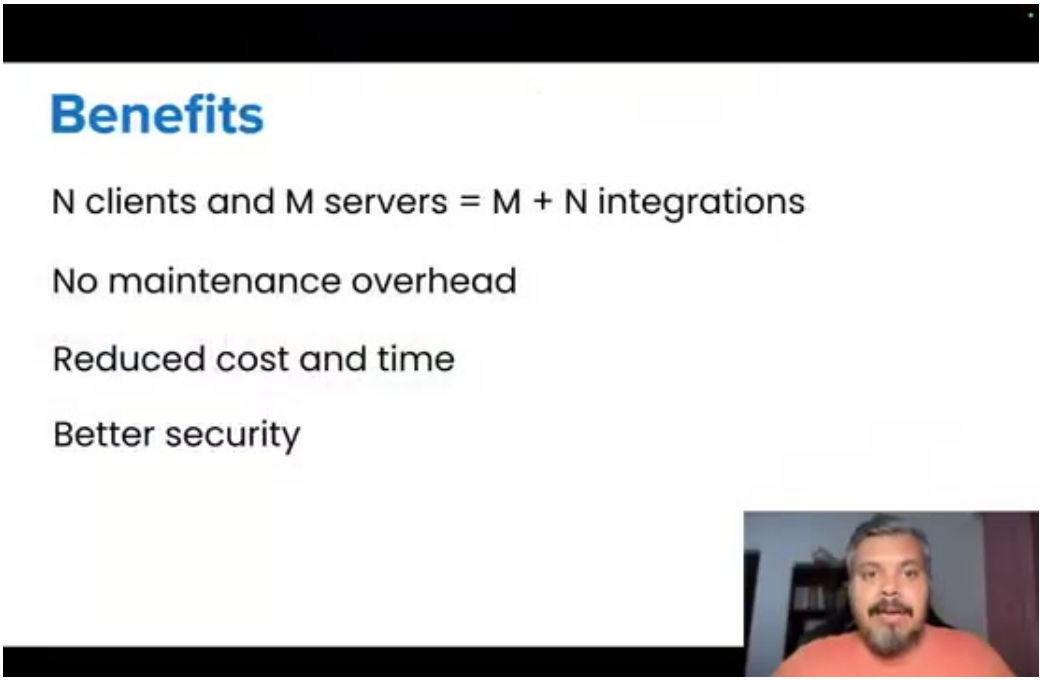

Figure 46. Frame at 46:58.

So these are the main benefits of MCP. Again, everything depends on a simple fact and the fact is that the entire heavy lifting is done by the server.

Section 47 — 47:08 – 47:19

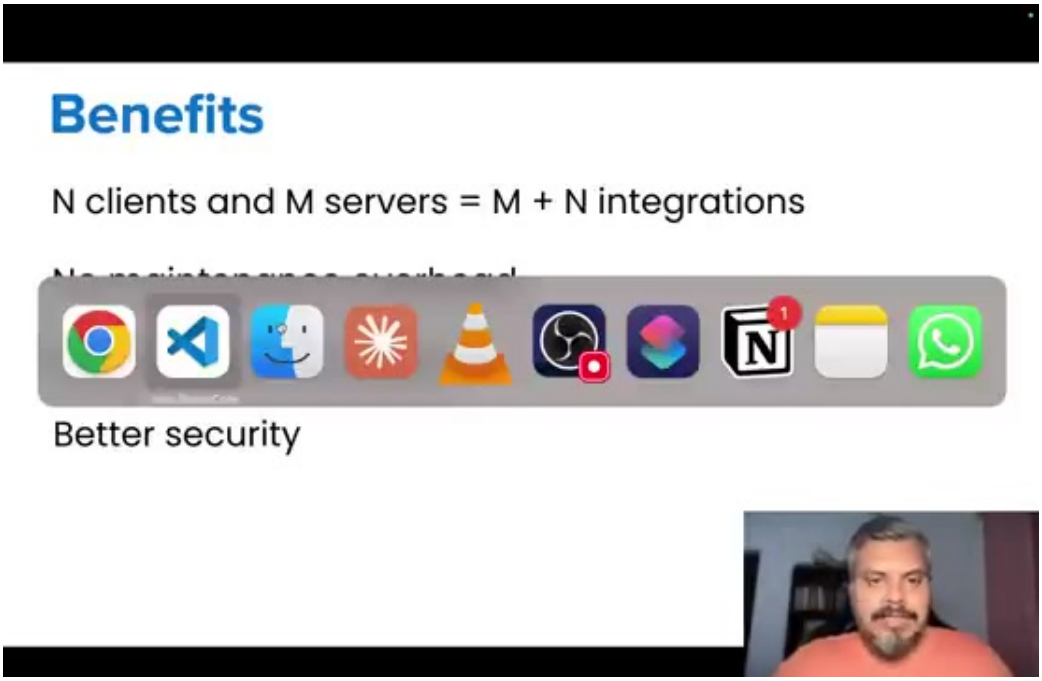

Figure 47. Frame at 47:18.

Again, everything depends on a simple fact and the fact is that the entire heavy lifting is done by the server. On the client side, the AI chatbot has to do nothing. You basically have to maintain a JSON file like this to connect to various servers.

Section 48 — 47:19 – 47:19

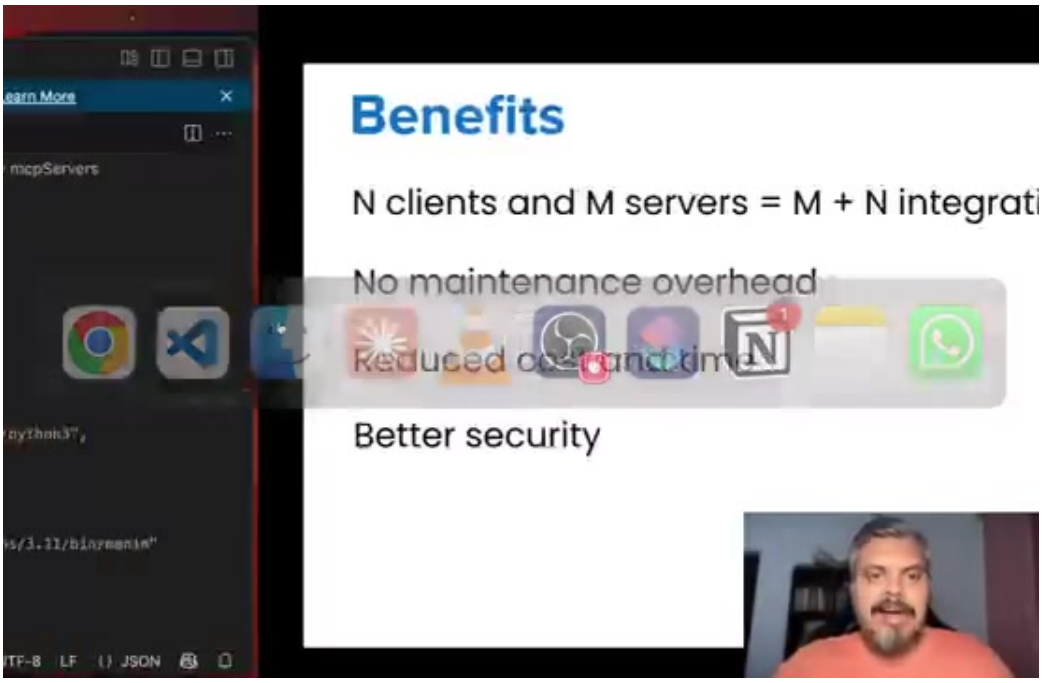

Figure 48. Frame at 47:19.

You basically have to maintain a JSON file like this to connect to various servers.

Section 49 — 47:19 – 47:23

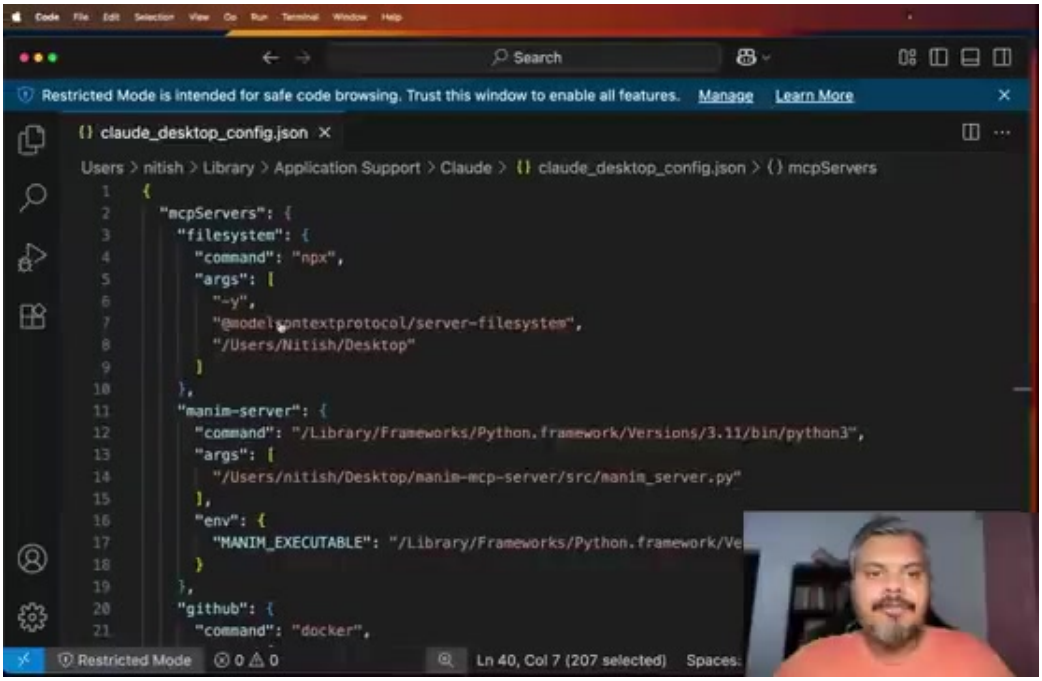

Figure 49. Frame at 47:19.

You basically have to maintain a JSON file like this to connect to various servers.

Section 50 — 47:23 – 52:00

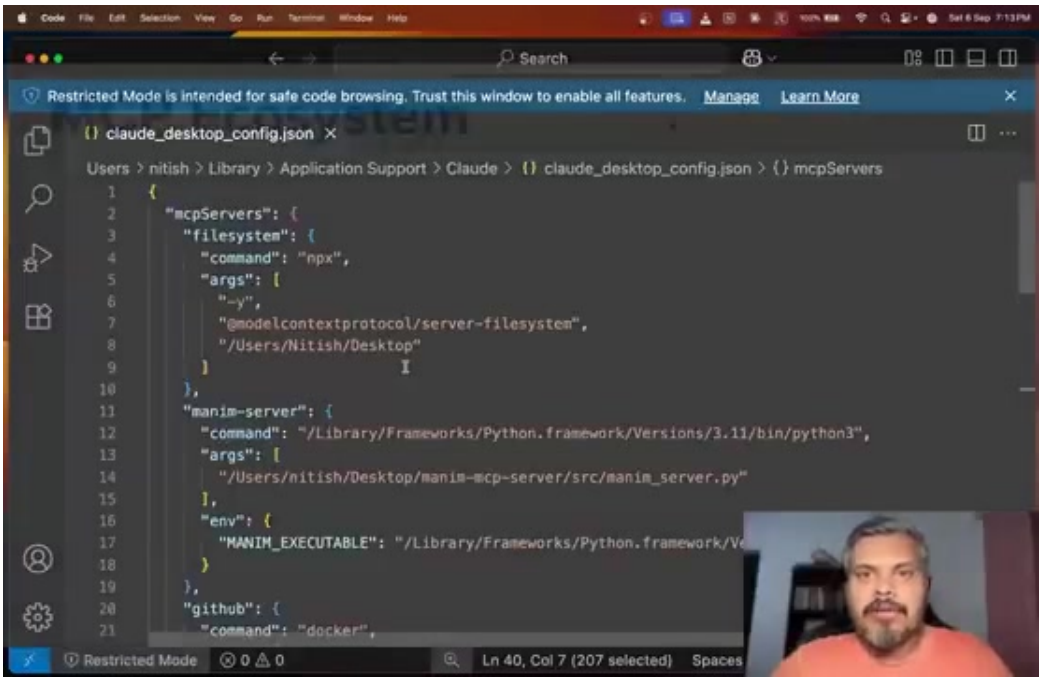

Figure 50. Frame at 47:27.

You basically have to maintain a JSON file like this to connect to various servers. And that's it. Your work is done. So I have given you an overview of how MCP works. But now I want to discuss one more thing with you. And it is that how is MCP becoming so popular? What is the reason that MCP's ecosystem is growing so fast? And why does it feel that MCP will become an industry standard in the next 3 to 5 years? Now the reason behind it is very simple. You can easily understand this whole dynamic. So what is happening is that some very famous AI chatbots are openly saying that we support MCP. Claude Dextop said that we support MCP. Cursor, Windsurf, Perplexity also said that we support MCP. Now as soon as these chatbots said that we support MCP, so a lot of the services started getting pressure on them. Services like GitHub, Slack, Google Drive, etc. It started getting pressure because it shows these companies that whatever people will do in the future, they will do all the work through these AI tools. So it won't be like before that if I want to access a document in Drive, then I will open Drive.google.com and read the document there. Instead people will simply say on their chatbots that give me this document from my Drive from this folder. So they also know that in the future, whatever traction they will have, whatever users will come, they will come through these AI tools. So if these AI tools are supporting MCP, then I should also make MCP servers from my APIs. Because if I make MCP servers once, then any MCP compliant, compatible tool can easily connect with my server. It doesn't need to write a custom code. So what is happening in this? All your famous services are becoming MCP servers. Now the fun part is that the more MCP servers you make, the more the MCP ecosystem will benefit. Yesterday a company made a new AI chatbot. It simply has to do that it has to make its AI chatbot a MCP-compatible client. And if it makes it, then it can connect thousands of MCP servers on day one without writing any piece of code. So the basic funda is that more AI clients are coming, more AI chatbots are coming, as a client, so a lot of servers are being made. A lot of servers are being made, so the pressure on new chatbots is that they should connect with these servers. So the new clients are automatically MCP compliant. So basically a network effect is being made, because of which the ecosystem is growing exponentially. So more adoption, more standardization, more value of the ecosystem. And if a client or a server, basically an AI chatbot or a tool, if MCP is not adopting it, then it means that the massive ecosystem will be cut off. And it will have to write a lot of custom code in order to make sure that its AI chatbot interacts with all the services properly. Which will be stupid again. That is why MCP has positioned himself in the right way. And it seems that in the next 3 to 5 years it will become an industry standard. So with that, I will conclude this video. I really hope that I was able to explain to you in a story, why MCP is needed. I started from day one. And today's scenario, I brought you to that point. I really hope that the video was long, but not boring. And you were able to derive some value. Now in the next video, in very detail, at the architecture level, we will understand how MCP works. The Watt of MCP. So I really hope you are excited for the next video. If you liked this video, please like it. If you have not subscribed to this channel, please do subscribe. See you in the next video. Bye.